

DBMS Interview Questions - SDE Interview Prep

PART 1: TOP 10 MOST IMPORTANT QUESTIONS

Q1: Explain the ACID properties in DBMS

Answer: ACID stands for Atomicity, Consistency, Isolation, and Durability in a DBMS these are those properties that ensure a safe and secure way of sharing data among multiple users.

- **Atomicity:** This property reflects the concept of either executing the whole query or executing nothing at all, which implies that if an update occurs in a database then that update should either be reflected in the whole database or should not be reflected at all.
- **Consistency:** This property ensures that the data remains consistent before and after a transaction in a database.
- **Isolation:** This property ensures that each transaction is occurring independently of the others. This implies that the state of an ongoing transaction doesn't affect the state of another ongoing transaction.
- **Durability:** This property ensures that the data is not lost in cases of a system failure or restart and is present in the same state as it was before the system failure or restart.

Polished Answer: ACID properties are the four fundamental guarantees that ensure reliable transaction processing in a DBMS:

- **Atomicity:** A transaction is treated as a single indivisible unit. Either all operations complete successfully or none do. If any part fails, the entire transaction rolls back.
- **Consistency:** The database transitions from one valid state to another. All constraints, cascades, and triggers must be satisfied before a transaction commits.
- **Isolation:** Concurrent transactions execute as if they were running sequentially. Intermediate states of one transaction are invisible to others, preventing interference.
- **Durability:** Once committed, transaction changes persist permanently, even through system crashes or power failures, typically via write-ahead logging.

TL;DR: ACID = Atomicity (all-or-nothing), Consistency (valid state transitions), Isolation (transactions don't interfere), Durability (committed data survives failures).

Keywords: Transaction properties, Atomicity, Consistency, Isolation, Durability, Data integrity,

Q2: Explain Normalization and the different normal forms (1NF, 2NF, 3NF, BCNF)

Answer: Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It divides large tables into smaller related tables and establishes relationships between them.

- **1NF (First Normal Form):** Ensures that the data in the table is atomic, meaning each column contains indivisible values (no multi-valued attributes).
- **2NF (Second Normal Form):** 2NF is achieved when the table is in 1NF and all non-key attributes are fully dependent on the primary key.
- **3NF (Third Normal Form):** A table is in 3NF if it is in 2NF and there is no transitive dependency (non-key attributes depend on other non-key attributes).
- **BCNF (Boyce-Codd Normal Form):** A stricter version of 3NF where every determinant is a candidate key.

Polished Answer: Normalization is a systematic database design technique that eliminates data redundancy and prevents update anomalies by decomposing tables into smaller, well-structured relations:

- **1NF:** Removes repeating groups. Every column holds atomic (single) values. Example: Instead of storing "Math, Science, English" in one column, create separate rows.
- **2NF:** Builds on 1NF. Every non-key attribute must depend on the entire primary key, not just part of it. This mainly applies to composite keys. Example: If StudentID + CourseID is the key, InstructorName depends only on CourseID--move it to a Course table.
- **3NF:** Builds on 2NF. Eliminates transitive dependencies--non-key attributes shouldn't depend on other non-key attributes. Example: DeptName depends on DeptID, not directly on EmpID--move it to a Department table.
- **BCNF:** A stricter 3NF where every functional dependency's determinant must be a candidate key. Handles edge cases where 3NF still allows redundancy.

TL;DR: Normalization = organizing data to eliminate redundancy. 1NF = atomic values, 2NF = no partial dependencies, 3NF = no transitive dependencies, BCNF = every determinant is a candidate key.

Keywords: Database design, Redundancy elimination, Functional dependencies, Transitive dependency, Data anomalies, Table decomposition

Q3: What is the difference between DELETE, TRUNCATE, and DROP?

Answer:

- **DELETE:** Deletes specific rows from a table based on a condition. It logs each row deletion and can be rolled back inside a transaction. It is a slower operation because it deletes rows one by one and triggers any associated triggers.
- **TRUNCATE:** Removes all rows from a table without logging individual row deletions. It cannot be rolled back and is faster than DELETE. It does not delete rows individually and triggers are not fired.
- **DROP:** Removes the entire table, including its structure and data. Once a table is dropped, it cannot be recovered easily.

Polished Answer: These three SQL commands differ significantly in what they remove and their reversibility:

- **DELETE:** A DML command that removes specific rows using a WHERE clause. It's row-by-row, logged in the transaction log, fires triggers, and can be rolled back. Best for selective deletion.
- **TRUNCATE:** A DDL command that removes ALL rows at once. It deallocates entire data pages rather than individual rows, making it much faster. It doesn't fire triggers and typically can't be rolled back. Table structure and indexes remain.
- **DROP:** A DDL command that removes the entire table--structure, data, indexes, and constraints. Nothing remains. Use only when the table is no longer needed.

TL;DR: DELETE = removes specific rows (slow, rollback-able), TRUNCATE = removes all rows (fast, can't rollback), DROP = removes entire table (structure + data).

Keywords: DML vs DDL, Row deletion, Table removal, Transaction log, Rollback capability, Performance comparison

Q4: Explain JOINS in SQL with types

Answer: A JOIN in SQL is an operation that combines columns from two or more tables

based on a related column between them. Types include:

- **INNER JOIN:** Returns only the rows where there is a match in both tables.
- **LEFT JOIN (LEFT OUTER JOIN):** Returns all rows from the left table and the matching rows from the right table. If there is no match, NULL values will be returned for columns from the right table.
- **RIGHT JOIN (RIGHT OUTER JOIN):** Returns all rows from the right table and the matching rows from the left table.
- **FULL JOIN (FULL OUTER JOIN):** Returns all rows when there is a match in either the left or the right table.
- **CROSS JOIN:** Returns the Cartesian product of two tables.
- **SELF JOIN:** A join where a table is joined with itself.

Polished Answer: JOINS are the foundation of relational database queries, combining data from related tables:

- **INNER JOIN:** The intersection of both tables. Only matching rows appear. Most commonly used.
- **LEFT JOIN:** Preserves all rows from the left table. Unmatched right-side rows become NULL. Essential when you need complete data from the primary table.
- **RIGHT JOIN:** Mirror of LEFT JOIN--preserves all rows from the right table. Often can be rewritten as a LEFT JOIN.
- **FULL OUTER JOIN:** Union of both tables. Includes all rows from both, with NULLs where there's no match. Useful for finding orphaned records in either table.
- **CROSS JOIN:** Cartesian product--every row from one table paired with every row from another. Dangerous with large tables; rarely used except for generating combinations.
- **SELF JOIN:** Joining a table with itself using aliases. Common for hierarchical data like employee-manager relationships.

TL;DR: JOINS combine tables. INNER = matching only, LEFT/RIGHT = all from one side + matches, FULL = all from both, CROSS = all combinations, SELF = table joined with itself.

Keywords: Table relationships, Outer joins, Cartesian product, NULL handling, Query optimization, Data retrieval

Q5: What is the difference between a primary key and a unique key?

Answer:

- **Primary Key:** Uniquely identifies each record in a table. Cannot contain NULL values. A table can have only one primary key. It is usually used for the main identifier.
- **Unique Key:** Ensures that all values in a column are unique across all rows. Can contain NULL values (depending on the database). A table can have multiple unique keys.

Polished Answer: Both ensure uniqueness, but they serve different roles:

- **Primary Key:** The primary identifier of a row. Cannot be NULL (enforces both NOT NULL and UNIQUE). Only one per table. Often used as a foreign key reference in other tables. Typically clustered index in most databases.
- **Unique Key:** Enforces uniqueness but allows NULL values (one NULL in most databases, unlimited in MySQL). Multiple unique keys allowed per table. Used to enforce business rules like unique email addresses or phone numbers.

TL;DR: Primary Key = unique + NOT NULL + only one per table. Unique Key = unique + allows NULL + multiple allowed.

Keywords: Row identification, NULL handling, Table constraints, Data integrity, Foreign key references, Index creation

Q6: What is an index? Explain clustered vs non-clustered index

Answer: An index is a data structure that improves the speed of data retrieval operations on a database table. It works like a table of contents in a book.

- **Clustered Index:** Organizes the data in the table according to the index. There can only be one clustered index per table because the data rows can only be sorted one way.
- **Non-clustered Index:** Creates a separate structure from the table that holds pointers to the actual data rows. Multiple non-clustered indexes can be created on a table.

Polished Answer: An index is a database object that dramatically speeds up data retrieval by creating a searchable structure:

- **Clustered Index:** Determines the physical storage order of rows in a table. The table itself is stored as the leaf nodes of the index. Only one per table (data can only be physically sorted one way). Often created on the primary key. Searching is fast because finding the index entry means finding the data.
 - **Non-clustered Index:** A separate structure that stores sorted key values and pointers (row locators) to the actual data. Multiple allowed per table (SQL Server allows 999). Slightly
-

slower than clustered because it requires an extra lookup step to reach the data after finding the index entry.

TL;DR: Index = faster data retrieval. Clustered = physical order (1 per table), Non-clustered = separate pointer structure (multiple allowed).

Keywords: Query performance, B-tree structure, Data retrieval speed, Physical vs logical storage, Index design, Lookup operations

Q7: What is the difference between WHERE and HAVING?

Answer: WHERE is used to filter rows before any grouping or aggregation happens in the query. It works on individual records and cannot use aggregate functions. HAVING is applied after GROUP BY and is meant for filtering aggregated results.

Polished Answer: The key distinction lies in when filtering occurs during query execution:

- **WHERE:** Applied FIRST, before grouping. Filters individual rows based on column values. Cannot use aggregate functions (COUNT, SUM, AVG). Use WHERE to reduce the dataset before aggregation--this improves performance.
- **HAVING:** Applied AFTER grouping. Filters groups based on aggregate results. Can use aggregate functions. Use HAVING only when you need to filter based on aggregated values.

Execution order: FROM WHERE GROUP BY HAVING SELECT ORDER BY

Example: Filtering orders by status goes in WHERE, but filtering customers with total orders greater than 5 goes in HAVING.

TL;DR: WHERE = filters rows before grouping (no aggregates), HAVING = filters groups after aggregation (uses aggregates).

Keywords: Query execution order, Row filtering, Group filtering, Aggregate functions, Query optimization, SQL clauses

Q8: What is the difference between DBMS and RDBMS?

Answer:

- **DBMS:** Stores data as files or in non-relational form. Does not support relationships between data. Does not enforce integrity constraints. Examples: Microsoft Access, XML

Database.

- **RDBMS:** Stores data in tables (rows and columns). Supports relationships using foreign keys. Enforces integrity using primary and foreign keys. Examples: MySQL, Oracle, SQL Server.

Polished Answer: The fundamental difference is how data is organized and related:

- **DBMS:** A general-purpose system for storing and retrieving data. Data may be stored in files, hierarchical structures, or other non-relational formats. No concept of relationships between data entities. No built-in integrity enforcement. Limited support for concurrent access and complex queries. Examples: file systems, XML databases, early hierarchical systems.
- **RDBMS:** A specialized DBMS based on the relational model. Data is organized into tables with rows and columns. Relationships between tables are explicitly defined using foreign keys. Enforces integrity constraints (primary keys, foreign keys, unique constraints). Supports ACID transactions and complex SQL queries with JOINS. Examples: MySQL, PostgreSQL, Oracle, SQL Server.

TL;DR: DBMS = stores data as files (no relationships), RDBMS = stores data in related tables (foreign keys, integrity constraints).

Keywords: Relational model, Data organization, Foreign keys, Integrity constraints, Table structure, SQL support

Q9: Explain different types of relationships in DBMS

Answer: The three main types of relationships in DBMS are:

- **One-to-One (1:1):** A record in one table is associated with a single record in another table.
- **One-to-Many (1:M):** A record in one table is associated with multiple records in another table.
- **Many-to-One (M:1):** Multiple records in Table A are associated with a single record in Table B.
- **Many-to-Many (M:M):** Multiple records in one table are associated with multiple records in another table.

Polished Answer: Database relationships define how entities interact:

- **One-to-One (1:1):** Each row in Table A relates to exactly one row in Table B. Example: One person has one passport. Implemented by making the foreign key unique in one table.

- **One-to-Many (1:M):** One row in Table A relates to multiple rows in Table B. Example: One customer can place many orders. Implemented by placing a foreign key in the "many" side table.
- **Many-to-One (M:1):** The reverse of One-to-Many. Multiple rows in Table A relate to one row in Table B. Example: Many students belong to one department.
- **Many-to-Many (M:M):** Multiple rows in both tables relate to each other. Example: Students enroll in courses--each student takes multiple courses, each course has multiple students. Implemented using a junction/bridge table with foreign keys to both tables.
- **Self-Referencing:** A row relates to another row in the same table. Example: Employee has a manager who is also an employee.

TL;DR: 1:1 = one row to one row, 1:M = one row to many rows, M:1 = many rows to one row, M:M = many rows to many rows (requires junction table).

Keywords: Entity relationships, Foreign keys, Junction tables, Cardinality, Database design, Referential integrity

Q10: What are constraints in DBMS? Explain types with examples

Answer: Constraints in DBMS are rules that limit the type of data that can be inserted into a table to ensure data integrity and consistency. Common types include:

- **NOT NULL:** Ensures that a column cannot have NULL values. Example: `Name VARCHAR(50) NOT NULL;`
- **PRIMARY KEY:** Uniquely identifies each record in a table. Example: `ID INT PRIMARY KEY;`
- **FOREIGN KEY:** Ensures referential integrity between tables. Example: `BranchCode INT FOREIGN KEY REFERENCES Branch (BranchCode);`
- **UNIQUE:** Ensures all values in a column are distinct. Allows NULL. Example: `Email VARCHAR(100) UNIQUE;`
- **CHECK:** Ensures values satisfy a condition. Example: `Age INT CHECK (Age >= 18);`
- **DEFAULT:** Assigns a default value if none provided. Example: `Status VARCHAR(10) DEFAULT 'Active';`

Polished Answer: Constraints are database rules that enforce data quality and integrity at the database level:

- **NOT NULL:** Prevents NULL values in a column. Essential for required fields like names, dates, and identifiers.
- **PRIMARY KEY:** Combines NOT NULL and UNIQUE. Uniquely identifies each row. Only one per table. Automatically creates an index.
- **FOREIGN KEY:** Enforces referential integrity by linking a column to a primary key in another table. Prevents orphaned records. Supports cascade actions (ON DELETE CASCADE, ON UPDATE CASCADE).
- **UNIQUE:** Enforces distinct values but allows NULLs. Multiple per table. Useful for email, phone, username fields.
- **CHECK:** Validates values against a condition before insertion/update. Example: ensuring age > 18, salary > 0.
- **DEFAULT:** Provides a fallback value when none is specified during insertion. Useful for status fields, timestamps.

TL;DR: Constraints = rules for data integrity. NOT NULL (no NULLs), PRIMARY KEY (unique identifier), FOREIGN KEY (referential link), UNIQUE (distinct values), CHECK (condition), DEFAULT (fallback value).

Keywords: Data integrity, Data validation, Table rules, Referential integrity, Database design, Input validation

PART 2: QUESTIONS 11-25

Q11: Explain the Three-Schema Architecture in DBMS

Answer: The Three-Schema Architecture is a DBMS design that separates the database into three levels to provide data abstraction and data independence.

- **External Level (View Level):** Defines how different users view the database. Each user sees only the data relevant to them.
- **Conceptual Level (Logical Level):** Describes the overall logical structure of the database, including tables, relationships, and constraints.
- **Internal Level (Physical Level):** Describes how data is physically stored on disk, including file organization and indexing.

Polished Answer: The Three-Schema Architecture is a fundamental DBMS design principle that provides data abstraction and independence:

- **External/View Level:** The highest level. Each user or application group sees a customized view of the database tailored to their needs. Multiple views can exist. This provides security by hiding sensitive data.
- **Conceptual/Logical Level:** The middle level. Describes the complete logical structure--all tables, relationships, constraints, and data types. Independent of both physical storage and user views. This is what database designers work with.
- **Internal/Physical Level:** The lowest level. Describes how data is physically stored--file organization, indexing techniques, storage allocation. Managed entirely by the DBMS, hidden from users and designers.

The key benefit is **data independence**: changes at one level don't affect other levels. Logical independence means changing the conceptual schema doesn't affect external views.

Physical independence means changing storage doesn't affect the logical schema.

TL;DR: Three levels = External (user views), Conceptual (logical structure), Internal (physical storage). Provides data abstraction and independence.

Keywords: Data abstraction, Data independence, Database architecture, View level, Logical level, Physical level

Q12: What is a transaction in DBMS? Explain transaction states

Answer: A transaction in DBMS is a sequence of one or more SQL operations executed as a single unit of work. A transaction ensures data integrity, consistency, and isolation, and it guarantees that the database reaches a valid state, regardless of errors or system failures.

Polished Answer: A transaction is a logical unit of work that must be atomic--either fully completed or fully rolled back. Transaction states include:

- **Active:** The transaction is executing its operations.
- **Partially Committed:** All operations executed successfully but not yet written to permanent storage. A failure here still allows rollback.
- **Committed:** All changes permanently saved to the database. Cannot be undone.
- **Failed:** An error occurred. The transaction cannot proceed normally.
- **Aborted:** The transaction has been rolled back to the state before it started. Database restored to its prior consistent state.

Transactions are managed by the Transaction Manager and are essential for maintaining ACID properties in concurrent environments.

TL;DR: Transaction = unit of work (all-or-nothing). States: Active Partially Committed Committed (success) or Active Failed Aborted (failure).

Keywords: ACID properties, Transaction states, Atomicity, Concurrent execution, Data consistency, Rollback

Q13: What is the difference between a stored procedure and a function?

Answer: A stored procedure is a precompiled collection of one or more SQL statements stored in the database. Stored procedures allow users to execute a series of operations as a single unit. A stored function is a set of SQL statements that can be executed in the database. It accepts input parameters, performs some logic, and returns a value. Functions must return a value.

Polished Answer: Both are reusable database objects, but with key differences:

- **Return Value:** Functions MUST return a single value (scalar or table). Stored procedures may return zero, one, or multiple result sets; they don't strictly "return" a value but use OUT parameters.
- **Usage in Queries:** Functions can be used inline in SELECT, WHERE, and JOIN clauses. Stored procedures cannot be used in queries; they're called using CALL or EXEC.
- **Transaction Control:** Stored procedures can use COMMIT and ROLLBACK. Functions typically cannot manage transactions.
- **Parameters:** Functions mainly use IN parameters. Stored procedures support IN, OUT, and INOUT parameters.
- **Purpose:** Stored procedures are for performing actions (INSERT, UPDATE, DELETE operations). Functions are for computation and returning values.

TL;DR: Functions must return a value (usable in queries), stored procedures perform actions (may not return values, can't be used in queries).

Keywords: Stored procedures, User-defined functions, Reusability, Database logic, Return values, Query usage

Q14: What is a trigger? How does it differ from a stored procedure?

Answer: A trigger is a special kind of stored procedure that automatically executes (or "fires") in response to certain events on a table, such as insertions, updates, or deletions. Triggers are used to enforce business rules, maintain consistency, or log changes.

Polished Answer: Key differences between triggers and stored procedures:

- **Invocation:** Triggers fire AUTOMATICALLY when a specified event occurs (INSERT, UPDATE, DELETE). Stored procedures must be called EXPLICITLY by a user or application.
- **Parameters:** Triggers cannot accept parameters. Stored procedures can accept input parameters and return output.
- **Transaction Context:** Triggers execute within the transaction that fired them and can participate in its rollback. Stored procedures run as independent units.
- **Use Cases:** Triggers are used for auditing, maintaining derived data, enforcing complex business rules. Stored procedures are used for reusable business logic and batch operations.
- **Timing:** Triggers can be BEFORE or AFTER an event. Stored procedures run when invoked.

TL;DR: Triggers = automatic on table events (no params), Stored Procedures = manual invocation (with params).

Keywords: Event-driven execution, Database automation, Auditing, Business rules, BEFORE/AFTER triggers, DML operations

Q15: What is a deadlock? How can it be prevented?

Answer: A deadlock occurs when two or more transactions are blocked because each transaction is waiting for the other to release resources. This results in a situation where none of the transactions can proceed.

Prevention Techniques:

- **Lock ordering:** Ensuring that all transactions acquire locks in the same predefined order.
- **Timeouts:** Automatically rolling back transactions that have been waiting too long for resources.
- **Deadlock detection:** Periodically checking for deadlocks and aborting one of the transactions to break the cycle.

Polished Answer: A deadlock is a circular waiting condition. Transaction A holds lock on

Resource 1 and wants Resource 2; Transaction B holds Resource 2 and wants Resource 1--neither can proceed.

DBMS handles deadlocks through:

- **Deadlock Detection:** The system maintains a wait-for graph. If a cycle is found, a deadlock exists. The DBMS selects a "victim" transaction (often the one that has done the least work) and rolls it back.
- **Deadlock Prevention:** Ensures deadlocks cannot occur:
- **Timeouts:** Transactions that wait too long are automatically aborted.
- **Conservative approach:** Transaction acquires ALL needed locks upfront.

Deadlock Avoidance: Uses algorithms like wait-die or wound-wait to decide whether a transaction should wait or abort based on timestamps.

TL;DR: Deadlock = circular waiting (A waits for B, B waits for A). Prevented by lock ordering, timeouts, or detection with victim rollback.

Keywords: Circular waiting, Resource contention, Lock ordering, Victim selection, Concurrency control, Transaction rollback

Q16: Explain the difference between a superkey, candidate key, and primary key

Answer:

- **Superkey:** A set of one or more attributes that can uniquely identify a row in a table. It may contain unnecessary attributes.
- **Candidate Key:** A minimal superkey that uniquely identifies a row, with no redundant attributes. A table can have multiple candidate keys.
- **Primary Key:** One candidate key chosen to be the main identifier for the table.

Polished Answer: These keys form a hierarchy of uniqueness:

- **Superkey:** ANY set of attributes (or combination) that uniquely identifies each row. Can include extra, unnecessary attributes. Example: {StudentID}, {StudentID, Name}, {StudentID, Email, Phone} are all superkeys--only {StudentID} is minimal.
- **Candidate Key:** A MINIMAL superkey--removing any attribute destroys its uniqueness. Example: {StudentID} and {Email} might both be candidate keys if each uniquely identifies students.
- **Primary Key:** The candidate key chosen by the designer as the main identifier. Only one

per table. Other candidate keys become alternate keys.

- **Composite Key:** A candidate key made of two or more attributes. Example: {OrderID, ProductID} together form a composite key in an order details table.

TL;DR: Superkey (any unique set) Candidate Key (minimal unique set) Primary Key (chosen candidate key).

Keywords: Key hierarchy, Uniqueness constraints, Minimal superkey, Composite keys, Database design, Record identification

Q17: What is a subquery? Explain correlated vs non-correlated subquery

Answer: A subquery in SQL is a query embedded within another query. It is used to retrieve data that will be used in the outer query.

- **Non-correlated subquery:** The inner query executes once, independent of the outer query, and its result is used by the outer query.
- **Correlated subquery:** The inner query depends on the outer query and executes once for each row in the outer query.

Polished Answer: Subqueries (nested queries) come in two main types:

- **Non-correlated (Simple) Subquery:** The inner query executes ONCE and returns a value or set of values. The outer query uses these results. Efficient because the inner query runs only once.

Correlated Subquery: The inner query references columns from the outer query and executes ONCE PER ROW of the outer query. Can be slow on large datasets.

- Example: Find employees earning more than their department average:

```
SELECT Name
FROM Employee E1 WHERE Salary > (SELECT AVG(Salary) FROM Employee
E2 WHERE E2.DeptID = E1.DeptID);
```

Performance Note: Non-correlated subqueries are generally faster. Correlated subqueries can often be rewritten as JOINS for better performance.

TL;DR: Subquery = query inside query. Non-correlated = inner runs once, Correlated = inner runs per outer row (slower).

Keywords: Nested queries, Inner query, Outer query, Performance optimization, Scalar subquery, Row-by-row execution

Q18: What are aggregate functions in SQL?

Answer: Aggregate functions in SQL are functions that operate on a set of values (or a group of rows) and return a single result. They are often used in conjunction with the GROUP BY clause.

Common aggregate functions:

- **COUNT():** Returns the number of rows or non-NULL values.
- **SUM():** Returns the sum of values in a numeric column.
- **AVG():** Returns the average value of a numeric column.
- **MAX():** Returns the maximum value in a column.
- **MIN():** Returns the minimum value in a column.

Polished Answer: Aggregate functions collapse multiple rows into a single summary value. They're essential for reporting and data analysis:

- **COUNT(*):** Counts all rows (including NULLs). `COUNT(column)` counts non-NULL values only. `COUNT(DISTINCT column)` counts distinct non-NULL values.
- **SUM():** Adds all values in a numeric column. Ignores NULLs. Use with DISTINCT for unique values.
- **AVG():** Calculates the arithmetic mean. Ignores NULLs. Returns NULL if no non-NULL values exist.
- **MAX()/MIN():** Find the largest/smallest value. Work with numbers, strings (alphabetical), and dates (chronological).
- **GROUP_CONCAT()/STRING_AGG():** Concatenates values from multiple rows (database-specific).

Common patterns:

- With GROUP BY: `SELECT DeptID, AVG(Salary) FROM Employee GROUP BY DeptID;`
- With HAVING: `SELECT DeptID, COUNT(*) FROM Employee GROUP BY DeptID HAVING COUNT(*) > 10;`

TL;DR: Aggregate functions = compute summary from multiple rows. COUNT, SUM, AVG, MAX, MIN. Used with GROUP BY and HAVING.

Keywords: Data summarization, GROUP BY clause, HAVING clause, Data analysis, NULL handling, Reporting queries

Q19: What is a VIEW? How does it differ from a table?

Answer: A View is a virtual table created by querying one or more base tables. It does not store data physically but dynamically retrieves it when queried. Unlike a table, a view does not store its own data but presents data from other tables.

Polished Answer: A view is a saved SQL query that acts like a virtual table:

- **Storage:** Tables store data physically. Views store only the query definition--data is fetched from underlying tables when the view is accessed.
- **Performance:** Tables are fast (direct data access). Views may be slower (query execution overhead on every access).
- **Data Updates:** Tables allow direct INSERT, UPDATE, DELETE. Views have limitations--simple views on single tables are updateable, but complex views (with JOINS, aggregations) are not.
- **Use Cases:**
 - Providing a security layer (show only specific columns/rows)
 - Maintaining consistent interfaces
 - Achieving logical data independence

Types of Views:

- **Simple View:** Based on a single table, no functions or groups.
- **Complex View:** Based on multiple tables or includes functions/groups.
- **Materialized View:** Physically stores result data (unlike regular views).

TL;DR: View = virtual table (stored query), Table = physical storage. Views simplify queries and provide security but don't store data.

Keywords: Virtual tables, Query abstraction, Security layer, Data retrieval, Logical independence, Materialized views

Q20: What is the role of a Database Administrator (DBA)?

Answer: A Database Administrator (DBA) is responsible for managing and overseeing the entire database environment. Key responsibilities include:

- **Database Design:** Structuring the database for optimal storage and performance.
- **Backup and Recovery:** Ensuring regular backups and providing recovery solutions.

- **Performance Tuning:** Monitoring and optimizing the database's performance.
- **Security Management:** Managing user access, privileges, and enforcing security policies.
- **Data Integrity:** Ensuring data consistency and integrity through constraints and checks.
- **Upgrades and Patches:** Keeping the database software up-to-date.
- **Troubleshooting:** Identifying and resolving database-related issues.

Polished Answer: The DBA is the primary custodian of the database environment:

- **Database Design and Architecture:** Collaborates with developers to design schema, establish relationships, and define constraints for optimal performance and maintainability.
- **Backup and Recovery:** Develops backup strategies (full, incremental, differential), tests recovery procedures, and ensures point-in-time recovery capability.
- **Performance Monitoring and Tuning:** Monitors query performance, identifies bottlenecks, optimizes indexes, and tunes database parameters.
- **Security Administration:** Creates and manages user accounts, assigns roles and privileges (RBAC), implements encryption, and audits access.
- **Capacity Planning:** Forecasts storage and compute needs, plans for scaling.
- **High Availability:** Configures replication, clustering, and failover mechanisms to ensure uptime.
- **Compliance and Documentation:** Ensures regulatory compliance (GDPR, HIPAA) and maintains documentation.

TL;DR: DBA = manages database lifecycle. Responsibilities: design, backup/recovery, performance tuning, security, integrity, upgrades, troubleshooting.

Keywords: Database management, Backup strategy, Performance optimization, Access control, Data integrity, Disaster recovery

Q21: What is referential integrity? Explain with example

Answer: Referential Integrity ensures that relationships between tables are maintained correctly. It requires that the foreign key in one table must match a primary key or a unique key in another table (or be NULL). This ensures that data consistency is maintained, and there are no orphan records in the database.

Example: In the Orders table, if the CustomerID is a foreign key, it should match a valid CustomerID in the Customers table or be NULL.

Polished Answer: Referential integrity is a fundamental relational database concept that

ensures foreign key values are valid:

- **Rule:** Every foreign key value must reference an existing primary key value in the parent table, OR be NULL (if the foreign key column allows NULLs).
- **Enforcement:** The DBMS automatically checks this constraint on every INSERT, UPDATE, and DELETE operation.
- **Cascade Actions:**
 - ON DELETE SET NULL: Set child's foreign key to NULL when parent is deleted.
 - ON DELETE RESTRICT: Prevent parent deletion if child rows exist.
 - ON UPDATE CASCADE: Update child's foreign key when parent's key changes.

Example: An Orders table has Customer ID as a foreign key. You cannot insert an order with CustomerID = 999 if no customer with ID 999 exists. Similarly, you cannot delete a customer who has orders (unless CASCADE is specified).

TL;DR: Referential integrity = foreign keys must match primary keys. Prevents orphaned records. Supports CASCADE, SET NULL, RESTRICT actions.

Keywords: Foreign key constraints, Data consistency, Cascade actions, Orphan records, Parent-child tables, Data integrity

Q22: What is the difference between UNION and UNION ALL?

Answer:

- **UNION:** Combines the result of two queries and removes duplicate rows.
- **UNION ALL:** Combines the result of two queries but does not remove duplicates, thus it is faster than UNION.

Polished Answer: Both combine result sets from multiple queries, but differ in duplicate handling:

- **UNION:** Removes duplicate rows from the combined result. Requires sorting and comparison to eliminate duplicates, making it slower. The columns in both SELECT statements must match in number and compatible data types.
- **UNION ALL:** Simply appends all rows from both queries without deduplication. Much faster because it skips the duplicate-removal step. Use when you know there are no duplicates or you need to preserve all rows.

Performance:

- UNION = additional sort/distinct operation = more CPU and memory

- UNION ALL = simple concatenation = faster

Usage: Use UNION when you need distinct results. Use UNION ALL for performance, especially with large datasets where duplicates are not a concern.

TL;DR: UNION = combines + removes duplicates (slower), UNION ALL = combines without deduplication (faster).

Keywords: Set operations, Duplicate removal, Query performance, Result combination, Data deduplication, SQL optimization

Q23: Explain the concept of data independence in DBMS

Answer: Data independence allows changing the data structure without altering the composition of any of the executing application programs.

Polished Answer: Data independence is the ability to modify schema at one level without affecting the schema at the next higher level:

- **Logical Data Independence:** The ability to change the conceptual schema (logical structure) without changing external schemas (user views) or application programs.
- Harder to achieve because changes at the logical level can affect multiple views.

Physical Data Independence: The ability to change the internal schema (physical storage) without changing the conceptual schema.

- Example: Changing storage from HDD to SSD, or modifying file organization, shouldn't affect how tables are defined or queried.
- Easier to achieve because the DBMS handles the physical layer automatically.

Benefits:

- Application programs remain stable despite database changes
- Performance can be tuned without breaking functionality
- Easier maintenance and evolution of the database

TL;DR: Data independence = changing one schema level doesn't break other levels. Logical = logical changes don't affect views, Physical = storage changes don't affect logic.

Keywords: Schema evolution, Logical independence, Physical independence, Application stability, Database maintenance, Abstraction

Q24: What is a materialized view? When should it be used?

Answer: A materialized view is a database object that contains the results of a query. Unlike a regular view, which is a virtual table, a materialized view stores data physically, improving query performance by precomputing and storing results. Use Case: Commonly used in data warehousing and reporting systems where the same data is frequently queried.

Polished Answer: A materialized view is a precomputed query result stored as a physical table:

- **How it works:** The query is executed once and the results are stored. Subsequent queries read from the stored results instead of recomputing from base tables.
- **Refresh mechanisms:**
- **On-commit:** Automatically refreshed when base tables change.
- **Scheduled:** Refreshed at regular intervals.

Trade-offs:

- **Advantages:** Dramatically faster query performance for complex aggregations, joins, and reporting queries. Reduces load on base tables.
- **Disadvantages:** Data can become stale. Storage overhead. Refresh operation can be expensive and impact performance.

When to use: When read performance is critical and data freshness can tolerate delays. Ideal for daily/weekly reports, dashboards, and aggregated analytics.

TL;DR: Materialized view = physically stored query results. Fast reads, but data becomes stale. Use for reporting/analytics where freshness isn't critical.

Keywords: Query precomputation, Physical storage, Data freshness, Performance optimization, Data warehousing, Reporting queries

Q25: What is the difference between a B-tree and B+ tree?

Answer:

- **B-tree (Balanced Tree):** A self-balancing tree that maintains sorted data and allows searches, insertions, deletions in logarithmic time. Stores data in both internal and leaf nodes.
 - **B+ tree:** An extension of B-tree widely used in databases for indexing. Stores all records in
-

leaf nodes. Internal nodes store only keys and pointers. Has a linked list at the leaf level.

Polished Answer: B+ trees are the standard indexing structure in most modern databases.

Key differences:

- **Data Storage:**

- B+ tree: Stores data ONLY in leaf nodes. Internal nodes contain only keys for navigation.

Leaf Node Structure:

- B-tree: Leaf nodes are not linked.
- B+ tree: All leaf nodes are linked in a sequential linked list, enabling efficient range queries.

Tree Height:

- B+ tree is typically shorter (more branching possible at internal nodes), leading to fewer disk accesses.

Range Queries: B+ trees excel at range queries (WHERE age BETWEEN 20 AND 30) due to the leaf-level linked list.

Why B+ trees: Databases prefer B+ trees because they minimize disk I/O, support efficient range scans, and keep keys compact in internal nodes.

TL;DR: B+ tree = data only in leaves + linked leaf nodes (better for range queries). B-tree = data in all nodes.

Keywords: Index structure, Balanced trees, Disk I/O optimization, Range queries, Leaf nodes, Database indexing

PART 3: QUESTIONS 26-50

Q26: What is a cursor? When should it be used?

Answer: A cursor in DBMS is a pointer to a result set of a query. It allows for row-by-row processing of query results, which is useful when dealing with large datasets.

- **Implicit cursors:** Automatically created by the DBMS.
- **Explicit cursors:** Manually created by the programmer.

Polished Answer: A cursor enables row-by-row traversal of query results when set-based operations aren't suitable:

- **Implicit Cursors:** Auto-created for single-row operations. No user control over their lifecycle.
- **Explicit Cursors:** User-defined. Allow step-by-step processing: DECLARE OPEN FETCH (repeatedly) CLOSE.

Use Cases:

- Processing rows one at a time when set operations are impractical
- Row-by-row validation or business logic
- Iterating over hierarchical data with custom logic

Performance Warning: Cursors are inherently slower than set-based operations because:

- Each FETCH is a round-trip to the database
- Row-by-row processing adds overhead
- Locks may be held longer

Best Practice: Prefer set-based SQL operations whenever possible. Use cursors only when row-by-row logic is genuinely required.

TL;DR: Cursor = pointer to result set for row-by-row processing. Slower than set operations. Use only when necessary.

Keywords: Row processing, Result set traversal, Explicit vs implicit cursors, Performance overhead, Procedural SQL, FETCH operations

Q27: What are the different types of database locks?

Answer: Types of locks in DBMS:

- **Shared Lock (S Lock):** Allows multiple transactions to read data but prevents modification.
- **Exclusive Lock (X Lock):** Prevents any other transaction from reading or modifying the locked resource.
- **Intent Lock:** Signals that a transaction intends to lock a resource.
- **Update Lock (U Lock):** Used when a transaction intends to update a resource.

Polished Answer: Locks are concurrency control mechanisms that prevent data conflicts:

- **Shared Lock (S-Lock / Read Lock):**
 - Allows reading but prevents writing.
 - Compatible with other shared locks, incompatible with exclusive locks.
-

Exclusive Lock (X-Lock / Write Lock):

- Only one transaction can hold an exclusive lock on a resource.
- Allows both reading and writing (by the lock holder only).
- Incompatible with all other locks.

Update Lock (U-Lock):

- A transitional lock that allows reading (shared lock) but prevents deadlocks when a transaction intends to write.
- Converts to an exclusive lock when the update occurs.

Intent Locks:

- Signal intent to acquire locks at a finer granularity.
- Types: Intent Shared (IS), Intent Exclusive (IX), Shared with Intent Exclusive (SIX).
- Used in hierarchical locking systems.

Lock Compatibility:

- $S + S = \text{Compatible}$
- $S + X = \text{Incompatible}$
- $X + X = \text{Incompatible}$

TL;DR: Locks prevent conflicts. Shared = read-only (multiple allowed), Exclusive = read/write (one only), Update = transition lock, Intent = hierarchical signaling.

Keywords: Concurrency control, Lock compatibility, Read locks, Write locks, Lock granularity, Deadlock prevention

Q28: Explain the concept of concurrency control in DBMS

Answer: Concurrency control ensures that database transactions are executed in a way that prevents conflicts, such as data inconsistency, when multiple transactions are executed simultaneously.

Techniques:

- **Locking:** Transactions acquire locks on data to prevent conflicts.
 - **Timestamp Ordering:** Assigns timestamps to transactions to determine execution order.
 - **Optimistic Concurrency Control:** Transactions execute without locking, checking for conflicts at commit.
 - **Two-Phase Locking:** Involves growing (acquiring locks) and shrinking (releasing locks)
-

phases.

Polished Answer: Concurrency control manages simultaneous transaction execution to ensure serializable outcomes:

- **Locking Protocols:**

- Prevents dirty reads, lost updates, and other anomalies.

Two-Phase Locking (2PL):

- **Growing Phase:** Transactions acquire locks but don't release any.
- **Shrinking Phase:** Transactions release locks but don't acquire new ones.
- Ensures serializability.

Timestamp Ordering:

- Each transaction gets a unique timestamp.
- Transactions execute in timestamp order.
- Older transactions get priority over newer ones (wait-die, wound-wait schemes).

Optimistic Concurrency Control (OCC):

- Transactions execute without acquiring locks.
- At commit time, the system validates for conflicts.
- Suitable for low-contention environments.

Multi-Version Concurrency Control (MVCC):

- Maintains multiple versions of data.
- Readers see a consistent snapshot without blocking writers.
- Used by PostgreSQL, Oracle, MySQL InnoDB.

Problems prevented: Dirty reads, non-repeatable reads, phantom reads, lost updates.

TL;DR: Concurrency control = managing simultaneous transactions. Techniques: Locking, 2PL, Timestamp Ordering, OCC, MVCC. Prevents data inconsistencies.

Keywords: Serializability, Locking protocols, Two-phase locking, Transaction isolation, MVCC, Data consistency

Q29: What is denormalization? When should you denormalize?

Answer: Denormalization is the process of combining tables to improve query performance, often by introducing redundancy. Normalization minimizes redundancy, denormalization sacrifices some of it to improve speed for read-heavy operations.

Polished Answer: Denormalization is the deliberate introduction of redundancy for performance gains:

- **What:** Combining normalized tables into fewer tables, or adding duplicate columns to avoid JOINS.
- **Why:** JOINS become expensive as data grows. Denormalized tables allow direct access without multiple table lookups.
- **When to Denormalize:**
- **Reporting/analytics:** Complex aggregations across normalized tables.
- **Caching frequently accessed data:** Storing computed values alongside source data.
- **Reducing JOIN complexity:** When queries consistently require the same multi-table joins.

Trade-offs:

- **Advantages:** Faster reads, simpler queries, fewer joins.
- **Disadvantages:** Increased storage, potential data inconsistency, more complex updates (must update redundant data everywhere).

Denormalization Strategies:

- Adding redundant columns
- Combining tables
- Using materialized views (often considered a denormalization tool)

TL;DR: Denormalization = adding redundancy for read performance. Use for read-heavy systems. Downside: storage increase and update complexity.

Keywords: Query optimization, Data redundancy, Read performance, Write trade-offs, Table combination, Data consistency

Q30: What are the different types of backups in DBMS?

Answer: Types of backups:

- **Full Backup:** Copies the entire database. Most comprehensive but takes time and storage.
- **Incremental Backup:** Copies only data changed since the last backup.
- **Differential Backup:** Copies all changes since the last full backup.
- **Transaction Log Backup:** Copies the transaction log for point-in-time recovery.

Polished Answer: A robust backup strategy typically combines multiple backup types:

- **Full Backup:**
- Serves as the baseline for other backup types.
- Slowest to create but fastest to restore (from a single backup).

Incremental Backup:

- Captures only changes since the last backup (full or incremental).
- Fastest to create, minimal storage.
- Slowest to restore (must apply full + all incrementals in order).

Differential Backup:

- Captures all changes since the last FULL backup.
- Faster than full, slower than incremental.
- Faster restore than incremental (only need full + latest differential).

Transaction Log Backup:

- Captures the transaction log entries.
- Enables point-in-time recovery (e.g., restore to 2:30 PM yesterday).
- Typically used in conjunction with full/differential backups.

Restore strategies:

- Full only: Simple, but limited to backup time.
- Full + Differential: Restore full, then latest differential.
- Full + Incremental chain: Restore full, then all incrementals in sequence.

TL;DR: Full = complete copy, Incremental = changes since last backup, Differential = changes since last full, Transaction Log = enables point-in-time recovery.

Keywords: Backup strategy, Data recovery, Point-in-time recovery, Storage management, Disaster recovery, Restore process

Q31: What is data redundancy? How can it be reduced?

Answer: Data redundancy refers to the unnecessary repetition of data in a database. It can lead to inconsistencies, increased storage requirements, and maintenance challenges.

Reduction methods:

- **Normalization:** Splitting large tables into smaller ones to eliminate redundancy.
- **Eliminating duplicate data:** Using constraints like UNIQUE and PRIMARY KEY to enforce data consistency.

Polished Answer: Data redundancy is the storage of the same data multiple times:

- **Problems Caused:**
- **Storage waste:** Unnecessary disk space consumption.
- **Update anomalies:** Updating duplicate data requires multiple updates, increasing error risk.
- **Insertion anomalies:** Cannot insert data without other unrelated data.
- **Deletion anomalies:** Deleting data unintentionally removes other data.

Reduction Techniques:

- **Normalization:** Decompose tables to eliminate redundancy (1NF, 2NF, 3NF, BCNF).
- **Proper primary keys:** Ensure each entity type is stored once.
- **Foreign key usage:** Store relationships via references, not duplicating data.
- **Constraints:** Use UNIQUE constraints to prevent duplicate entries.
- **Database design reviews:** Regularly audit schema for redundancy.

Note: Some controlled redundancy may be acceptable in denormalized designs for performance reasons.

TL;DR: Data redundancy = repeated data. Problems: inconsistency, storage waste, anomalies. Solution: normalization and proper key usage.

Keywords: Data duplication, Normalization, Update anomalies, Storage efficiency, Data consistency, Database design

Q32: Explain the concept of a schema in DBMS

Answer: A schema in DBMS is the structure that defines the organization of data in a database. It includes tables, views, relationships, and other elements. A schema defines the tables and their columns, along with the constraints, keys, and relationships.

Polished Answer: A schema is the blueprint of the database:

- **What it includes:**
- Relationships between tables (foreign keys)
- Constraints (primary keys, unique, check, etc.)
- Views, indexes, stored procedures, functions
- Access privileges and permissions

Schema vs. Data:

- Schema is the structure (changed rarely).
- Data/instance is the actual content (changes frequently).
- Schema is defined using DDL; data is manipulated using DML.

Types of Schemas:

- **Physical Schema:** How data is stored on disk.
- **Logical Schema:** How data is organized logically (tables, relationships).
- **External Schema:** How users view the data (views).

Schema Design: Good schema design follows normalization principles to minimize redundancy and ensure integrity.

TL;DR: Schema = database blueprint (tables, columns, relationships, constraints). Changed via DDL. Data is the actual content changed via DML.

Keywords: Database structure, DDL commands, Database design, Logical schema, Physical schema, Data organization

Q33: What is a transaction log?

Answer: A transaction log is a record that keeps track of all transactions executed on a database. It ensures that changes made by transactions are saved, and in case of a system failure, the log can be used to recover the database to its last consistent state.

Polished Answer: The transaction log is a critical component for database durability and recovery:

- **What it contains:**
- Before and after images of modified data
- Timestamps and transaction IDs
- Information about every data modification

Purpose:

- **Durability:** Ensures committed transactions survive system failures.
- **Rollback:** Provides data needed to undo uncommitted changes.
- **Recovery:** Enables restoration to a consistent state after crashes.
- **Replication:** Some systems use the log for replication to standby servers.

Write-Ahead Logging (WAL):

- Changes are written to the log **BEFORE** they're applied to the database.

- If a crash occurs, the log is replayed to restore consistency.
- Ensures the database can always recover to a consistent state.

Recovery Process:

- **UNDO:** Rollback incomplete transactions found in the log.
- **REDO:** Reapply committed transactions not yet written to disk.

TL;DR: Transaction log = sequential record of all changes. Enables durability, rollback, and crash recovery. Uses Write-Ahead Logging.

Keywords: Write-ahead logging, Crash recovery, Durability, Point-in-time recovery, UNDO/REDO, Database consistency

Q34: What is the purpose of the GROUP BY clause?

Answer: The GROUP BY clause is used in SQL to group rows that have the same values in specified columns into summary rows, often with aggregate functions like COUNT, SUM, AVG, MIN, or MAX. It is typically used to organize data for reporting or analysis.

Polished Answer: GROUP BY transforms individual rows into grouped summaries:

- **Basic Usage:** Groups rows by one or more columns. All rows with the same value in the grouped column(s) become one output row.
- **With Aggregate Functions:** After grouping, aggregate functions (COUNT, SUM, AVG, MIN, MAX) compute a single value per group.
- **Multiple Columns:** Can group by multiple columns for finer granularity.
- **HAVING Clause:** Filters groups after aggregation, based on aggregate results.

Execution Order: FROM WHERE GROUP BY HAVING SELECT ORDER BY LIMIT

Example:

sql

```

javascript.snippet
JAVASCRIPT

SELECT Department, COUNT(*) AS EmployeeCount, AVG(Salary) AS AvgSalary
FROM Employees
WHERE Status = 'Active'
GROUP BY Department
HAVING COUNT(*) > 1
ORDER BY AvgSalary DESC;

```

Rules:

- Columns in SELECT (not in aggregates) must appear in GROUP BY.

- WHERE filters rows before grouping; HAVING filters groups after aggregation.

TL;DR: GROUP BY = combines rows by common values for aggregation. Works with COUNT, SUM, AVG, MIN, MAX. HAVING filters groups.

Keywords: Data grouping, Aggregation, Reporting queries, GROUP BY syntax, Summary data, SQL analysis

Q35: Explain the concept of hashing in DBMS

Answer: Hashing in DBMS is used to map data (such as a key) to a fixed-size value or address, using a hash function. It is primarily used for quick data retrieval, particularly in hash indexes or hash tables.

How it works:

- A hash function takes the key and calculates a hash value.
- This hash value determines the bucket or slot where data is stored.
- When searching, the hash function is applied again to find the corresponding bucket.

Polished Answer: Hashing provides constant-time ($O(1)$) data access for equality lookups:

- **Hash Function:**
- Should distribute keys uniformly across buckets.
- Consistent: same key always yields same hash.

Hash Table Structure:

- An array of buckets.
- Each bucket holds one or more records.

Collision Handling:

- **Chaining:** Buckets contain linked lists of colliding records.
- **Open Addressing:** Find the next available bucket (linear probing, quadratic probing).

Types of Hashing:

- **Static Hashing:** Fixed number of buckets. Doesn't adapt to data growth.
- **Dynamic Hashing:** Number of buckets grows/shrinks with data (extendable hashing).

Advantages: Extremely fast for equality searches (WHERE id = 123).

Limitations: Not suitable for range queries (WHERE id BETWEEN 100 AND 200). B-trees are better for ranges.

TL;DR: Hashing = mapping keys to buckets via hash function. $O(1)$ lookup for equality

searches. Not for range queries.

Keywords: Hash function, Bucket allocation, Collision handling, Static vs dynamic hashing, O(1) lookup, Hash index

Q36: What is data partitioning in DBMS?

Answer: Data partitioning is the process of dividing large datasets into smaller, more manageable segments (partitions) to improve performance, scalability, and availability.

Types of partitioning:

- **Horizontal Partitioning:** Divides data by rows.
- **Vertical Partitioning:** Divides data by columns.
- **Range Partitioning:** Divides based on a range of values.
- **Hash Partitioning:** Distributes based on a hash value.

Polished Answer: Partitioning breaks large tables into smaller, independently manageable pieces:

- **Horizontal Partitioning (Sharding):**
- Each partition contains a subset of rows with the same schema.
- Example: Orders partitioned by year (Orders_2023, Orders_2024).

Vertical Partitioning:

- Splits columns into separate tables.
- Frequently accessed columns in one table, rarely used columns in another.
- Example: Separating employee contact info from salary details.

Partitioning Strategies:

- **Range Partitioning:** Partition by a value range (dates, IDs).
- **List Partitioning:** Partition by specific values (region, category).
- **Hash Partitioning:** Distribute rows using a hash function for even distribution.
- **Composite Partitioning:** Combine multiple strategies.

Benefits:

- Improved query performance (partition pruning skips irrelevant partitions).
- Easier data management (archive/delete entire partitions).
- Better scalability (distribute partitions across servers).
- Higher availability (failure of one partition doesn't affect others).

TL;DR: Partitioning = splitting large tables for performance. Horizontal = by rows, Vertical = by columns, Range/Hash/List = partitioning strategies.

Keywords: Horizontal partitioning, Vertical partitioning, Sharding, Query performance, Scalability, Data management

Q37: Explain transaction isolation levels

Answer: Transaction isolation levels in SQL define how isolated a transaction is from other concurrent transactions. Different levels prevent different concurrency problems:

- **Read Uncommitted:** Lowest level, allows dirty reads.
- **Read Committed:** Prevents dirty reads.
- **Repeatable Read:** Prevents dirty reads and non-repeatable reads.
- **Serializable:** Highest level, prevents all concurrency problems.

Polished Answer: Isolation levels balance data consistency against concurrency performance:

- **Read Uncommitted (Level 0):**
- No protection against non-repeatable or phantom reads.
- Fastest but least reliable. Rarely used.

Read Committed (Level 1):

- Prevents dirty reads--only committed data is visible.
- Still allows non-repeatable reads and phantom reads.
- Default in many databases (PostgreSQL, SQL Server, Oracle).

Repeatable Read (Level 2):

- Prevents dirty reads and non-repeatable reads.
- Row locks held until transaction completes.
- Still allows phantom reads.
- Default in MySQL InnoDB.

Serializable (Level 3):

- Prevents all concurrency problems: dirty reads, non-repeatable reads, phantom reads.
- Transactions execute as if they were sequential.
- Highest consistency but lowest concurrency.

Concurrency Problems:

- **Dirty Read:** Reading uncommitted data.
- **Non-repeatable Read:** Same query returns different results within a transaction.
- **Phantom Read:** New rows appear in subsequent queries.

TL;DR: Isolation levels: Read Uncommitted Read Committed Repeatable Read Serializable. Higher isolation = more consistency, less concurrency.

Keywords: Isolation levels, Dirty reads, Non-repeatable reads, Phantom reads, Data consistency, Concurrency performance

Q38: What is the use of the WITH CHECK OPTION in SQL views?

Answer: The WITH CHECK OPTION is used when creating a view in SQL to ensure that any insert or update operation on the view must adhere to the conditions defined in the view's WHERE clause.

Polished Answer: WITH CHECK OPTION enforces view conditions on data modifications:

- **Purpose:** Prevents inserting or updating rows through a view that would not be visible in that view.
- **Example:** A view `ActiveStudents` shows only students with `Status = 'Active'`. With WITH CHECK OPTION, you cannot insert an inactive student or update a student to inactive status through this view.
- **Without CHECK OPTION:** Modifications can create rows that don't match the view's WHERE clause--the row disappears from the view immediately after insertion (invisible data).
- **Types:** WITH CASCADED CHECK OPTION (checks all underlying views) and WITH LOCAL CHECK OPTION (only checks the current view).

Example:

sql

```

javascript.snippet
JAVASCRIPT

CREATE VIEW ActiveStudents AS SELECT * FROM Students WHERE Status = 'Active' WITH CHECK
OPTION
;-- This will FAIL: INSERT INTO ActiveStudents (Name, Status) VALUES ('John', 'Inactive
');
```

TL;DR: WITH CHECK OPTION = prevents inserting/updating rows that wouldn't match the view's WHERE clause.

Keywords: View constraints, Data visibility, INSERT restrictions, UPDATE restrictions, View integrity, SQL views

Q39: What are stored functions in DBMS?

Answer: A stored function is a set of SQL statements that can be executed in the database. It accepts input parameters, performs some logic, and returns a value. Stored functions are similar to stored procedures but differ in that they must return a value.

Polished Answer: Stored functions are reusable database objects that compute and return values:

- **Characteristics:**

- Can be used in SQL queries (SELECT, WHERE, JOIN clauses).
- Deterministic (always return same result for same input) or non-deterministic.
- Compiled and stored in the database for repeated use.

Advantages:

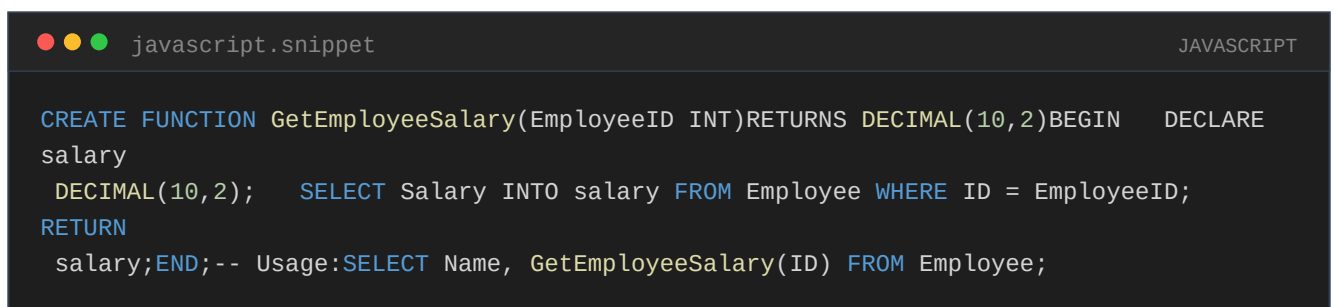
- **Code Reuse:** Write once, use in multiple queries.
- **Performance:** Executed on the database server, reducing network traffic.
- **Security:** Can restrict data access to only what the function returns.

Limitations:

- Cannot use COMMIT or ROLLBACK.
- Usually cannot modify data (some databases allow in functions, some don't).
- May have limited exception handling.

Example:

sql

A screenshot of a code editor window titled 'javascript.snippet' with a 'JAVASCRIPT' language indicator. The editor contains SQL code for creating a stored function named 'GetEmployeeSalary'. The code is as follows:

```
CREATE FUNCTION GetEmployeeSalary(EmployeeID INT) RETURNS DECIMAL(10,2) BEGIN
  DECLARE salary DECIMAL(10,2);
  SELECT Salary INTO salary FROM Employee WHERE ID = EmployeeID;
  RETURN salary;
END; -- Usage: SELECT Name, GetEmployeeSalary(ID) FROM Employee;
```

TL;DR: Stored function = reusable SQL code that returns a value. Usable in queries. Cannot manage transactions.

Keywords: User-defined functions, Return values, Code reusability, Database logic, Query usage, Performance

Q40: What is the difference between a trigger and a stored procedure?

Answer:

- **Trigger:** Automatically executes in response to events (INSERT, UPDATE, DELETE). Cannot be invoked manually. Tied to a specific event.
- **Stored Procedure:** Precompiled SQL statements executed explicitly. Invoked manually. Can accept input parameters.

Polished Answer: While both contain SQL logic, they differ fundamentally in invocation and behavior:

Trigger:

- **Invocation:** Automatic--fires when a specified event occurs.
- **Parameters:** Cannot accept parameters.
- **Transaction:** Executes within the same transaction as the firing event; can cause rollback.
- **Usage:** Auditing, enforcing business rules, maintaining derived data.
- **Timing:** Can be BEFORE, AFTER, or INSTEAD OF the event.

Stored Procedure:

- **Invocation:** Manual--called explicitly by users or applications.
- **Parameters:** Accepts IN, OUT, and INOUT parameters.
- **Transaction:** Runs as its own unit; can manage transactions (COMMIT/ROLLBACK).
- **Usage:** Reusable business logic, batch processing, complex operations.
- **Timing:** Executes when called.

Key Insight: Triggers are event-driven (reactive), while stored procedures are request-driven (proactive).

TL;DR: Trigger = automatic on table event (no params), Stored Procedure = manual call (with params). Triggers are event-driven, procedures are request-driven.

Keywords: Event-driven execution, Manual invocation, Parameters, Transaction handling, Auditing, Business logic

Q41: What is an Entity-Relationship Diagram (ERD)?

Answer: An ERD is a visual representation of the entities within a system and the relationships between those entities. It is used in database design to model the structure of data.

Components:

- **Entities:** Objects or things within the system (e.g., Student, Course).
- **Attributes:** Properties or details about an entity (e.g., Student Name, Course Duration).
- **Relationships:** How entities interact with each other (e.g., Student enrolls in Course).

Polished Answer: ERDs are the primary tool for conceptual database design:

- **Entities:** Represented as rectangles. Real-world objects, people, or concepts (Student, Course, Order). Weak entities depend on other entities for existence (OrderLine depends on Order).
- **Attributes:** Represented as ovals. Properties of entities. Types:
 - Composite: Can be divided (Full Name First Name, Last Name).
 - Multi-valued: Can have multiple values (Phone numbers).
 - Derived: Computed from other attributes (Age from DOB).
 - Key attributes: Uniquely identify the entity (underlined).

Relationships: Represented as diamonds. Connectivity:

- One-to-One (1:1)
- One-to-Many (1:M)
- Many-to-Many (M:M)
- Cardinality constraints (min-max notation)

ERD to Schema: ERDs are converted to relational schemas: entities become tables, attributes become columns, relationships become foreign keys.

TL;DR: ERD = visual database design. Entities (rectangles) + Attributes (ovals) + Relationships (diamonds). Converts to relational schema.

Keywords: Database modeling, Entity-relationship model, Database design, Cardinality, Conceptual modeling, Schema design

Q42: Explain the concept of data abstraction in DBMS

Answer: The process of hiding irrelevant details from users is known as data abstraction.

Levels:

- **Physical Level:** Lowest level. Data storage descriptions.
- **Conceptual/Logical Level:** What data is stored and relationships.
- **External/View Level:** Only part of the database, hides details.

Polished Answer: Data abstraction simplifies database interaction by hiding implementation details:

- **Physical Level (Internal):**
- File organization, indexing, compression.
- Details are hidden from developers and users.
- Managed entirely by the DBMS.

Logical Level (Conceptual):

- Describes WHAT data is stored.
- Tables, columns, relationships, constraints.
- Database designers work at this level.
- Independent of physical storage.

View Level (External):

- Describes what EACH USER SEES.
- Customized views showing only relevant data.
- Hides sensitive or irrelevant information.
- Multiple views can exist for different users.

Benefits: Simplification (users only see what they need), Security (sensitive data hidden), Flexibility (physical changes don't affect users).

TL;DR: Data abstraction = hiding complexity. Three levels: Physical (how stored), Logical (what stored), View (what users see).

Keywords: Abstraction levels, Data hiding, Physical storage, Logical structure, View level, Data security

Q43: What is the difference between intension and extension in a database?

Answer:

- **Intension (Schema):** The description/definition of the database. Specified during design. Remains mostly unchanged.
- **Extension (Snapshot):** The number of tuples present at any given time. Changes as data is created, updated, or deleted.

Polished Answer: These terms distinguish between database structure and content:

- **Intension (Schema):**
- Defines tables, columns, data types, constraints, relationships.
- Defined once during database design.
- Changes rarely (requires ALTER statements).
- Stored in the data dictionary/catalog.

Extension (Instance/Snapshot):

- The actual data stored in the database at a specific moment.
- Changes constantly as transactions insert, update, delete rows.
- The "current state" of the database.
- What you see when you query the database.

Analogy: Intension is like a blueprint of a building (fixed), while extension is like the current occupants and furniture (always changing).

TL;DR: Intension = database schema (structure, rarely changes). Extension = actual data snapshot (content, constantly changes).

Keywords: Database schema, Data snapshot, Structural definition, Data dictionary, Database state, Static vs dynamic

Q44: What is SQL injection? How can it be prevented?

Answer: SQL injection is a code injection technique where malicious SQL statements are inserted into input fields and executed by the database, potentially allowing attackers to access, modify, or delete data.

Polished Answer: SQL injection is a critical security vulnerability:

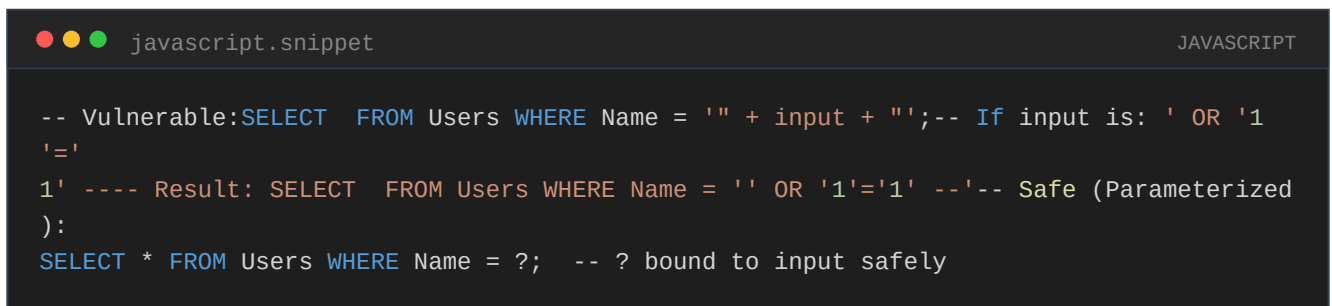
- **How it works:** Attackers exploit poorly validated user input by inserting SQL commands into form fields, query parameters, or URL inputs. Example: Input ' OR '1'='1 into a login field can bypass authentication.
 - **Impact:** Unauthorized data access, data modification, data deletion, full database
-

compromise.

- **Prevention techniques:**
- **Input Validation:** Validate and sanitize all user input.
- **Stored Procedures:** Encapsulate SQL logic and validate inputs.
- **Escaping Special Characters:** Escape single quotes and other SQL metacharacters.
- **Least Privilege:** Database users should have minimal necessary permissions.
- **ORM Usage:** Object-Relational Mappers often handle parameterization automatically.

Example:

sql



```
javascript.snippet                                JAVASCRIPT

-- Vulnerable:SELECT  FROM Users WHERE Name = ' + input + ' ;-- If input is: ' OR '1
'='
1' ---- Result: SELECT  FROM Users WHERE Name = ' OR '1'='1' --'-- Safe (Parameterized
):
SELECT * FROM Users WHERE Name = ?; -- ? bound to input safely
```

TL;DR: SQL injection = malicious SQL inserted via input. Prevention: parameterized queries, input validation, stored procedures, least privilege.

Keywords: Security vulnerability, Parameterized queries, Input sanitization, Prepared statements, Database security, Injection attacks

Q45: What is a B-tree? How is it used in databases?

Answer: A B-tree is a self-balancing tree data structure that maintains sorted data and allows searches, insertions, deletions in logarithmic time. B-trees are used in databases and file systems to store large amounts of data. All nodes can have multiple children, increasing search efficiency.

Polished Answer: B-trees are the workhorse of database indexing:

- **Structure:**
- All leaf nodes at the same level.
- Each node can have multiple children (unlike binary trees with only 2).
- Order (m) determines maximum children per node (typically 100+ in databases).

Key Properties:

- Sorted keys within each node.
-

- High branching factor reduces tree height.
- Shorter height = fewer disk accesses = faster searches.
- Logarithmic time complexity: $O(\log n)$ for search, insert, delete.

Why Databases Use B-trees:

- **Disk Optimization:** Each node fits in one disk block/page, making one disk read fetch many keys.
- **Balanced Height:** Predictable performance regardless of data distribution.
- **Range Queries:** Efficient sequential access.

Operations: Insertion and deletion may cause node splits and merges to maintain balance.

TL;DR: B-tree = self-balancing multi-way tree. Used for database indexing. High branching = fewer disk I/O = fast searches.

Keywords: Index structure, Balanced tree, Disk optimization, Multi-way branching, Logarithmic complexity, Database indexing

Q46: What is a composite key?

Answer: A composite key refers to a combination of two or more columns that can uniquely identify each tuple in a table. Example: studentId and firstname can be grouped to uniquely identify every tuple in the table.

Polished Answer: A composite key is a primary key or candidate key made of multiple columns:

- **Definition:** Two or more columns that together form a unique identifier for each row.
- **When to use:** When no single column can uniquely identify rows.
- **Example:** In an OrderDetails table, neither OrderID nor ProductID alone is unique. The combination (OrderID, ProductID) forms a composite key.
- **Relationship to candidate keys:** A composite key can be a candidate key or the chosen primary key.
- **Foreign keys referencing composite keys:** The referencing table must include all columns of the composite key.

Example:

sql

```
CREATE TABLE OrderDetails (    OrderID INT,    ProductID INT,    Quantity INT,    PRIMARY KEY (OrderID, ProductID));
```

TL;DR: Composite key = primary/candidate key made of 2+ columns. Used when single columns aren't unique enough.

Keywords: Multiple columns, Unique identification, Primary key, Candidate key, Table design, Composite identifier

Q47: What is the difference between a dense and sparse index?

Answer:

- **Dense Index:** Has an index entry for every record in the table.
- **Sparse Index:** Has entries for only some records (typically one per block).

Polished Answer: Dense and sparse indexes represent different index density strategies:

- **Dense Index:**
- Index entry contains key value and pointer to the actual record.
- Faster lookups--each key is directly indexed.
- Requires more storage space.
- Example: Non-clustered indexes in SQL databases.

Sparse Index:

- One index entry per block/page of records.
- Index entry points to the first record in each block.
- Slower lookups--may need to scan within a block after locating it.
- Uses less storage.
- Example: Clustered indexes where data blocks are indexed, not individual rows.

Combination: Databases often use sparse indexes at the upper levels of a B-tree and dense indexes at the leaf level.

TL;DR: Dense index = entry per row (fast, more storage). Sparse index = entry per block (slower, less storage).

Keywords: Index density, Storage efficiency, Lookup speed, Clustered indexes, Non-clustered indexes, Index structure

Q48: What is a covering index?

Answer: A covering index includes all the columns needed to satisfy a query, so the database doesn't need to access the table to retrieve additional data.

Polished Answer: A covering index is an index design optimization:

- **How it works:** The index contains all columns referenced in the query's SELECT, WHERE, and JOIN clauses. The database can satisfy the query entirely from the index, avoiding table access.
- **Performance benefit:** Eliminates the bookmark lookup (going from index to table), reducing I/O and improving query speed.
- **Example:** If a query is `SELECT Name, Email FROM Users WHERE Email = 'test@example.com'`, an index on `(Email, Name)` would be a covering index because both Email (WHERE) and Name (SELECT) are in the index.
- **Trade-off:** Covering indexes use more storage (duplicate data in the index). They also add overhead to INSERT, UPDATE, DELETE operations (must update the index).

TL;DR: Covering index = index contains all query columns. Query satisfied entirely from index. Faster but more storage and write overhead.

Keywords: Index optimization, Query performance, Index-only scan, Storage trade-offs, Query design, Index design

Q49: What is optimistic vs pessimistic concurrency control?

Answer:

- **Optimistic Concurrency Control:** Transactions execute without acquiring locks. Before committing, the system checks for conflicts.
- **Pessimistic Concurrency Control:** Transactions acquire locks upfront to prevent conflicts.

Polished Answer: These are two contrasting approaches to managing concurrent transactions:

- **Pessimistic (Lock-Based):**
 - Assumes conflicts are likely.
 - Blocks other transactions when data is locked.

- Guarantees no conflicts but reduces concurrency.
- Can cause deadlocks.
- Suitable for: High-contention environments where conflicts are frequent.

Optimistic (Validation-Based):

- Transactions execute WITHOUT locks.
- Before commit, the system VALIDATES whether conflicts occurred.
- If conflicts found, the transaction is rolled back.
- Assumes conflicts are rare.
- Higher concurrency when conflicts are infrequent.
- Suitable for: Low-contention environments (read-heavy workloads).

Hybrid Approaches: Many modern databases use MVCC (Multi-Version Concurrency Control), which combines elements of both.

TL;DR: Pessimistic = lock first, prevent conflicts. Optimistic = execute first, check for conflicts at commit. Choose based on contention levels.

Keywords: Lock-based control, Validation, Conflict detection, Concurrency strategy, Transaction management, Contention levels

Q50: What is the CAP theorem in database systems?

Answer: The CAP theorem (Brewer's theorem) states that a distributed database system can guarantee only two of three properties simultaneously:

- **Consistency:** All nodes see the same data at the same time.
- **Availability:** Every request receives a response (success or failure).
- **Partition Tolerance:** The system continues to operate despite network partitions.

Polished Answer: The CAP theorem is fundamental to understanding distributed database trade-offs:

- **Consistency:** After a write, all subsequent reads return the latest data. No node returns stale data.
- **Availability:** The system remains responsive even if some nodes fail. Every request gets a response.
- **Partition Tolerance:** The system functions correctly even when network partition occurs (nodes can't communicate with each other).

Key Insights:

- Partition tolerance is essential in distributed systems--partitions WILL happen.
- Therefore, systems must choose between consistency and availability during a partition.
- **CP Systems:** Favor consistency over availability. May reject requests during partitions to maintain consistency. Example: Banking systems.
- **AP Systems:** Favor availability over consistency. Continue serving requests with possibly stale data. Example: Social media feeds.

Database Classifications:

- **CP (Consistency + Partition Tolerance):** MongoDB, PostgreSQL with synchronous replication, HBase.
- **AP (Availability + Partition Tolerance):** Cassandra, DynamoDB, Couchbase.
- **CA (Consistency + Availability):** Theoretical only--no partition tolerance means single-node systems.

TL;DR: CAP = Consistency, Availability, Partition Tolerance. Distributed systems must choose 2 of 3. Since partitions are inevitable, the real choice is Consistency vs Availability.

Keywords: Distributed systems, Trade-offs, Network partitions, Data consistency, System availability, Database design

PART 4: QUESTIONS 51-100

Q51: What is query optimization?

Answer: Query optimization is the process where the DBMS determines the most efficient execution plan for a SQL query, considering factors like indexes, joins, and available resources.

Polished Answer: Query optimization is a critical DBMS function that transforms SQL into efficient execution:

- **Query Processing Pipeline:**
- **Translation:** Convert to internal representation (relational algebra).
- **Optimization:** Generate and select the best execution plan.
- **Execution:** Execute the chosen plan and return results.

Optimization Techniques:

- **Cost-based optimization:** Estimate costs of different plans based on statistics and

choose the cheapest.

- **Rule-based optimization:** Apply transformation rules (push down selects, reorder joins).
- **Index selection:** Choose appropriate indexes for data access.
- **Join ordering:** Determine optimal order of table joins.

What the optimizer considers:

- Table sizes and statistics
- Available indexes
- Join types (nested loop, hash join, merge join)
- Filtering selectivity
- I/O vs CPU costs

TL;DR: Query optimization = DBMS finds the most efficient execution plan. Uses cost estimates, statistics, and indexes to minimize resource usage.

Keywords: Execution plan, Cost-based optimization, Query processing, Index selection, Join ordering, Performance tuning

Q52: What is a heap file organization?

Answer: Heap file organization is a simple file organization method where records are stored in the order they are inserted, without any particular ordering or structure.

Polished Answer: Heap files are the simplest storage structure:

- **Characteristics:**
- No ordering, indexing, or clustering.
- Deleted records leave gaps that are reused.
- Full table scan required for searches.

Advantages:

- Fast insertion (no overhead to maintain order).
- Simple implementation.
- Suitable for small tables or temporary data.

Disadvantages:

- Linear search time: $O(n)$ for lookups.
- Slow for large tables.
- No optimization for range or equality queries.

Use cases: Temporary tables, staging tables, log tables where insertion performance matters

more than query speed.

TL;DR: Heap file = unordered record storage. Fast inserts, slow searches (full scan). Good for small/temporary tables.

Keywords: File organization, Record storage, Full table scan, Insertion performance, Search performance, Storage structure

Q53: Explain the difference between row-oriented and column-oriented storage

Answer:

- **Row-oriented storage:** Stores all attributes of a record together, organized by rows.
- **Column-oriented storage:** Stores all values of a specific attribute together, organized by columns.

Polished Answer: These are two fundamental data storage architectures:

- **Row-oriented (traditional RDBMS):**
- Reading a complete record is efficient.
- INSERT/UPDATE operations are fast (write one row).
- Scanning entire tables (all columns) is efficient.
- Best for: OLTP (transactional systems) where queries access specific rows.
- Examples: MySQL, PostgreSQL, SQL Server.

Column-oriented (analytical databases):

- Each column's data stored contiguously.
- Reading specific columns is efficient (only needed columns are read).
- Aggregations (SUM, AVG) on columns are very fast.
- INSERT/UPDATE operations are slower (must update multiple column stores).
- Best for: OLAP (analytical systems) with large-scale aggregations.
- Examples: Amazon Redshift, Google BigQuery, ClickHouse.

TL;DR: Row-oriented = data stored by rows (good for transactions). Column-oriented = data stored by columns (good for analytics/aggregations).

Keywords: Storage architecture, OLTP vs OLAP, Data compression, Query optimization, Aggregation performance, Storage layout

Q54: What is a checkpoint in DBMS?

Answer: A checkpoint is a point in time when the DBMS writes all modified data from buffers to disk, ensuring data consistency and reducing recovery time after crashes.

Polished Answer: Checkpoints are critical for efficient crash recovery:

- **What happens at a checkpoint:**
- A checkpoint record is written to the transaction log.
- The log sequence number (LSN) at the checkpoint is recorded.

Purpose:

- Reduce recovery time--recovery starts from the last checkpoint, not from the beginning.
- Limit the amount of transaction log that must be replayed.
- Provide a consistent point for backup and recovery.

Checkpoint frequency:

- Automatic: Based on time intervals or log size.
- Manual: Triggered by DBA.
- More frequent checkpoints = faster recovery but more I/O overhead.

Recovery using checkpoints: After a crash, the DBMS:

1. Restores the database state at the last checkpoint.
2. Replays (REDO) committed transactions after the checkpoint.
3. Rolls back (UNDO) uncommitted transactions.

TL;DR: Checkpoint = periodic buffer flush to disk. Reduces crash recovery time. Trade-off: more checkpoints = more I/O.

Keywords: Crash recovery, Buffer management, Dirty pages, Transaction log, REDO/UNDO, Checkpoint frequency

Q55: What is the difference between vertical and horizontal scaling?

Answer:

- **Vertical scaling:** Adding more resources (CPU, RAM, storage) to a single server.
 - **Horizontal scaling:** Adding more servers to distribute the load.
-

Polished Answer: These are two approaches to handling database growth:

- **Vertical Scaling (Scale-Up):**

- Simpler: no architectural changes needed.
- Limited by physical hardware limits and cost.
- Single point of failure.
- Examples: Upgrading from 16GB to 64GB RAM, adding faster SSDs.

Horizontal Scaling (Scale-Out):

- Adding more database servers/nodes.
- Distributes data and queries across multiple machines.
- Almost unlimited scalability.
- Requires data partitioning/sharding and replication strategies.
- Complex: needs distributed transaction handling, consistency management.
- Examples: Adding more nodes to a Cassandra cluster.

When to choose:

- Vertical: When data fits on one server, simpler management, lower complexity.
- Horizontal: When data is too large for one machine, high availability needed, global distribution required.

TL;DR: Vertical = bigger server (simple, limited). Horizontal = more servers (complex, scalable). Choose based on data volume and complexity tolerance.

Keywords: Scalability, Distributed systems, Server resources, Data partitioning, Load distribution, High availability

Q56: What is a distributed database?

Answer: A distributed database is a database that is spread across multiple interconnected computers, where data is stored and managed across these machines while appearing to the user as a single logical database.

Polished Answer: A distributed database system (DDBS) distributes data across multiple sites:

- **Characteristics:**
 - Connected via a network.
 - Each node has its own processing capability.
 - Data is distributed (partitioned or replicated) across nodes.
-

- Appears as a single logical database to users.

Distribution strategies:

- **Data Partitioning (Sharding):** Each node holds a portion of the data.
- **Data Replication:** Data copied to multiple nodes for availability and fault tolerance.
- **Hybrid:** Combination of partitioning and replication.

Advantages:

- Improved scalability (add more nodes).
- Better availability (node failure doesn't stop the entire system).
- Geographic distribution (data closer to users).

Challenges:

- Complexity in transaction management across nodes.
- Network latency.
- Consistency maintenance (per CAP theorem).
- Distributed query optimization.

TL;DR: Distributed database = data spread across multiple nodes, appears as one database.

Benefits: scalability, availability. Challenges: consistency, complexity.

Keywords: Data distribution, Sharding, Replication, Scalability, Availability, Distributed transactions

Q57: What is replication in databases?

Answer: Replication is the process of copying data from one database server (primary) to one or more other servers (replicas) to improve availability, fault tolerance, and read performance.

Polished Answer: Database replication is a core technique for high availability:

- **Types of Replication:**
- **Asynchronous Replication:** Primary commits without waiting for replicas. Faster but may lose recent data if primary fails.

Replication Topologies:

- **Master-Slave (Primary-Secondary):** Writes go to master; reads can go to replicas. Most common.
- **Multi-Master:** Multiple nodes can accept writes. Complex conflict resolution needed.

- **Cascading:** Replica replicates to other replicas. Reduces load on the master.

Benefits:

- **High Availability:** Replicas can take over if primary fails (failover).
- **Read Scalability:** Distribute read queries across replicas.
- **Disaster Recovery:** Geographic replication protects against regional failures.
- **Backup:** Can perform backups from replicas without impacting primary.

Challenges: Replication lag (stale data on replicas), conflict resolution in multi-master, complexity in failover.

TL;DR: Replication = copying data across servers. Benefits: availability, fault tolerance, read scaling. Challenges: lag, complexity.

Keywords: Data synchronization, Master-slave, Multi-master, Failover, Data availability, Read scalability

Q58: What is a cluster index? How does it differ from a non-clustered index?

Answer:

- **Clustered Index:** Determines physical order of data. Only one per table.
- **Non-clustered Index:** Separate structure with pointers to data. Multiple per table.

Polished Answer: The key distinction is in physical data organization:

- **Clustered Index:**
- The index IS the data (leaf nodes contain actual data).
- Only one per table (data can only be sorted one way).
- Changing the index requires physically rearranging data.
- Often created on primary key.
- Faster for queries that retrieve multiple related rows (range scans).

Non-clustered Index:

- Separate structure from the actual data.
 - Contains index keys and pointers (row locators) to data.
 - Multiple per table possible (SQL Server allows 999).
 - Data is not physically reordered.
 - Requires additional lookup (bookmark lookup) to retrieve data not in the index.
 - Faster for queries that return single rows (point lookups).
-

Performance: Clustered indexes are faster for range queries, non-clustered for point lookups with covering index optimization.

TL;DR: Clustered = data physically sorted (1 per table). Non-clustered = separate pointer structure (multiple allowed).

Keywords: Physical data order, Index pointers, Range queries, Point lookups, Query performance, Storage structure

Q59: What is the difference between logical and physical data independence?

Answer:

- **Logical Data Independence:** The ability to change the conceptual schema without changing external schemas or application programs.
- **Physical Data Independence:** The ability to change the internal schema without changing the conceptual schema.

Polished Answer: Both are types of data independence in the Three-Schema Architecture:

- **Logical Data Independence:**
- User views and applications remain unchanged.
- Example: Adding a new column to a table shouldn't break existing queries.
- Harder to achieve: logical changes can affect multiple views.

Physical Data Independence:

- Allows changing the PHYSICAL storage (file organization, storage devices, indexing).
- Logical structure remains unchanged.
- Example: Moving from HDD to SSD, or changing file organization, shouldn't affect table definitions.
- Easier to achieve: DBMS handles physical details automatically.

Benefits of both:

- Application programs remain stable despite database changes.
- Performance can be tuned without breaking functionality.
- Easier maintenance and evolution.

TL;DR: Logical independence = logical schema changes don't affect views. Physical independence = physical storage changes don't affect logical schema.

Keywords: Schema evolution, Application stability, Database maintenance, Data abstraction, Three-schema architecture, Independence levels

Q60: What is a recursive SQL query?

Answer: A recursive SQL query is a query that references itself to process hierarchical or recursive data structures, typically using Common Table Expressions (CTEs) with the RECURSIVE keyword.

Polished Answer: Recursive queries handle hierarchical data elegantly:

- **Structure:** Uses a Recursive CTE (WITH RECURSIVE) with two parts:
- **Recursive member:** References the CTE itself, processing one level deeper.

Syntax:

sql

```
javascript.snippet                                JAVASCRIPT

WITH RECURSIVE EmployeeHierarchy AS (              -- Anchor: Top-level managers    SELECT ID,
Name
, ManagerID, 0 AS Level                           FROM Employee WHERE ManagerID IS NULL      UNION ALL

-- Recursive: Next level down                      SELECT E.ID, E.Name, E.ManagerID, EH.Level + 1    FROM

Employee E    INNER JOIN EmployeeHierarchy EH ON E.ManagerID = EH.ID)SELECT * FROM
EmployeeHierarchy
;
```

- **Use cases:**
- Bill of materials (BOM) explosion
- Category and subcategory hierarchies
- Network/graph traversal

Termination: The recursion stops when the recursive member returns no rows.

TL;DR: Recursive query = self-referencing query using CTE with UNION ALL. Used for hierarchical data like org charts and category trees.

Keywords: Common Table Expressions, Hierarchical data, Recursive CTE, Tree traversal, Self-referencing queries, Data relationships

Q61: What is the difference between a temporary table and a view?

Answer:

- **Temporary Table:** A physical table created for temporary use, storing data. Exists only for the session or transaction.
- **View:** A virtual table defined by a query. Does not store data separately.

Polished Answer: Temporary tables and views serve different purposes:

- **Temporary Table:**
- Actually STORES data in a physical structure.
- Exists for the session (or transaction with `ON COMMIT DROP`).
- Can be indexed, modified, and have constraints.
- Data persists throughout the session.
- Faster for repeated access to the same intermediate result.

View:

- Created with `CREATE VIEW`.
- Does NOT store data--runs the underlying query each time.
- Exists until explicitly dropped.
- Cannot be indexed (except materialized views).
- Data is always current (reflects changes in base tables).
- Simplifies complex queries but adds query overhead.

When to use:

- Temporary table: When you need to store and reuse intermediate results multiple times.
- View: When you need a consistent interface to data without materializing it.

TL;DR: Temp table = physical storage, session-scoped, can index. View = virtual, always current, no storage.

Keywords: Session storage, Virtual tables, Intermediate results, Query optimization, Data persistence, Table types

Q62: What is a surrogate key?

Answer: A surrogate key is an artificial key generated by the database system to uniquely

identify a row, rather than using a natural business key. It has no business meaning.

Polished Answer: Surrogate keys are system-generated identifiers:

- **Characteristics:**

- Unique across the table.
- Never changes (unlike natural keys that might change).
- Typically numeric or UUID.

Advantages:

- **Stability:** Natural keys (email, SSN) can change; surrogate keys don't.
- **Performance:** Numeric keys are compact and efficient for indexing.
- **Simplicity:** No need to combine multiple attributes for uniqueness.
- **Consistency:** Uniform format regardless of business data.

Disadvantages:

- Not human-readable--users don't understand them.
- May need separate unique constraint on natural keys.
- Some argue they disassociate data from business context.

Example:

- Natural key: Email, SSN, EmployeeCode.
- Surrogate key: ID INT AUTO_INCREMENT PRIMARY KEY.

TL;DR: Surrogate key = artificial, system-generated identifier. Stable, performant, no business meaning. Example: auto-increment ID.

Keywords: Primary keys, Natural keys, Auto-increment, Unique identifiers, Database design, Key types

Q63: What is the difference between OLTP and OLAP?

Answer:

- **OLTP (Online Transaction Processing):** Systems optimized for handling many short, atomic transactions (INSERT, UPDATE, DELETE). Focus on data input and integrity.
- **OLAP (Online Analytical Processing):** Systems optimized for complex queries and data analysis (SELECT, aggregations). Focus on data retrieval and reporting.

Polished Answer: OLTP and OLAP are fundamentally different system types:

- **OLTP (Transactional):**

- **Operations:** INSERT, UPDATE, DELETE, simple SELECTs.
- **Data state:** Current, up-to-date operational data.
- **Schema:** Highly normalized (reduces redundancy, faster writes).
- **Concurrency:** High (many simultaneous users).
- **Response time:** Milliseconds.
- **Examples:** E-commerce transactions, banking systems, order management.

OLAP (Analytical):

- **Workload:** Fewer, complex, long-running queries.
- **Operations:** SELECT with aggregations, GROUP BY, complex JOINS.
- **Data state:** Historical, aggregated data (data warehouse).
- **Schema:** Often denormalized (star/snowflake schema).
- **Concurrency:** Low (few analytical users).
- **Response time:** Seconds to minutes.
- **Examples:** Business intelligence, reporting dashboards, trend analysis.

TL;DR: OLTP = transaction processing (fast writes, normalized, current data). OLAP = analytical queries (complex reads, denormalized, historical data).

Keywords: Transaction processing, Analytical processing, Data warehouse, Schema design, Workload optimization, System design

Q64: What is a two-phase commit protocol?

Answer: The two-phase commit (2PC) protocol is a distributed algorithm that ensures all nodes in a distributed system agree to commit or abort a transaction, ensuring atomicity across multiple nodes.

Polished Answer: Two-phase commit ensures distributed transaction consistency:

- **Phase 1: Prepare Phase:**
- Each participant performs the transaction and responds "ready" (can commit) or "abort" (cannot commit).
- Participants hold locks and log their prepared state.

Phase 2: Commit/Abort Phase:

- If ALL participants respond "ready," the coordinator sends "commit."
 - If ANY participant responds "abort" (or times out), the coordinator sends "abort."
 - Participants apply the decision and acknowledge.
-

Properties:

- Ensures atomicity across distributed nodes.
- Blocking protocol--if coordinator fails, participants may block (waiting for decision).

Challenges:

- Coordinator failure handling (needs recovery protocol).
- High overhead--multiple network round-trips.
- Not ideal for high-performance or high-latency scenarios.

TL;DR: 2PC = two-phase transaction protocol (prepare commit/abort). Ensures atomicity across distributed nodes. Blocking if coordinator fails.

Keywords: Distributed transactions, Atomicity, Prepare phase, Commit phase, Coordinator, Distributed systems

Q65: What is the difference between a database snapshot and a full backup?

Answer:

- **Database Snapshot:** A point-in-time read-only view of the database that captures the database state at a specific moment.
- **Full Backup:** A complete copy of the database stored as a separate file that can be used for restoration.

Polished Answer: Snapshots and backups serve different purposes:

- **Database Snapshot:**
- Uses copy-on-write technology--only stores changed pages after snapshot creation.
- Very fast to create (no data copy initially).
- Can be used for reporting, testing, or reverting to a previous state.
- Not a standalone backup--depends on the original database.
- Limited storage overhead for short-lived snapshots.

Full Backup:

- Creates a complete, physical copy of all data.
- Stored separately (disk, tape, cloud).
- Provides a restore point independent of the running database.
- Takes time and storage space.
- Cannot be used directly for querying--must be restored first.

Use Cases:

- Snapshot: Before schema changes, for reporting from a point-in-time state.
- Full Backup: Regular disaster recovery, compliance requirements.

TL;DR: Snapshot = instant point-in-time view (copy-on-write). Full backup = complete separate copy for disaster recovery.

Keywords: Point-in-time recovery, Copy-on-write, Disaster recovery, Data protection, Backup strategy, Database states

Q66: What is a multi-version concurrency control (MVCC)?

Answer: MVCC is a concurrency control method that maintains multiple versions of data items to allow readers to see consistent snapshots without blocking writers.

Polished Answer: MVCC is the foundation of modern database concurrency:

- **How it works:**
- When a transaction modifies data, it creates a NEW VERSION (not overwrite).
- Old versions are retained for transactions still reading them.
- Readers never block writers; writers never block readers.

Key benefits:

- **High concurrency:** Readers don't block writers and vice versa.
- **Consistent reads:** No dirty reads--readers see committed snapshot.
- **No read locks needed.**

Trade-offs:

- Storage overhead--multiple versions of data must be stored.
- Garbage collection--old versions must be periodically cleaned (VACUUM in PostgreSQL).
- Write amplification--updates create new versions.

Databases using MVCC: PostgreSQL, Oracle, MySQL (InnoDB), SQL Server (with READ_COMMITTED_SNAPSHOT).

TL;DR: MVCC = multiple data versions. Readers see snapshots without blocking writers. High concurrency, storage overhead.

Keywords: Concurrency control, Data versions, Snapshot isolation, Non-blocking reads, Transaction isolation, Write amplification

Q67: What is the role of a query processor?

Answer: The query processor is the DBMS component responsible for interpreting and executing SQL queries. It consists of a parser, optimizer, and executor.

Polished Answer: The query processor transforms SQL into actual data operations:

- **Parser (Syntax Analysis):**

- Creates a parse tree.
- Validates table names, column names, and permissions.

Optimizer (Query Optimization):

- Generates multiple execution plans.
- Estimates cost of each plan (I/O, CPU, memory).
- Selects the lowest-cost plan.
- Consider indexes, join strategies, and data statistics.

Executor (Execution Engine):

- Executes the optimized plan.
- Coordinates index scans, table scans, and joins.
- Returns results to the user.

Additional responsibilities:

- Query caching (store recently used query results).
- Permission checking.
- Constraint enforcement.

TL;DR: Query processor = parses SQL optimizes execution plan executes plan.

Components: Parser, Optimizer, Executor.

Keywords: SQL parsing, Execution plans, Query optimization, Cost estimation, Query execution, Database engine

Q68: What is a data dictionary?

Answer: A data dictionary is a repository of metadata about the database--information about tables, columns, constraints, indexes, and other database objects.

Polished Answer: The data dictionary is the database's "metadata repository":

- **Contents:**
- Constraints (primary keys, foreign keys, unique, check)
- Indexes and their definitions
- Views and their underlying queries
- Stored procedures, functions, and triggers
- User accounts and privileges
- Statistics used by the query optimizer

Characteristics:

- Maintained automatically by the DBMS.
- Updated when schema changes occur (DDL operations).
- Queryable by users (e.g., INFORMATION_SCHEMA in MySQL).

Uses:

- Database administration (understanding structure).
- Query optimization (statistics for cost estimation).
- Documentation generation.
- Impact analysis (what views use which tables).

TL;DR: Data dictionary = metadata repository. Stores table definitions, constraints, indexes, users. Queryable and auto-maintained.

Keywords: Metadata, Schema information, Database catalog, INFORMATION_SCHEMA, Database administration, Query optimization

Q69: What is the difference between a natural key and a surrogate key?

Answer:

- **Natural Key:** A key derived from real-world business data (e.g., email, SSN, employee code).
- **Surrogate Key:** An artificial key generated by the system (e.g., auto-increment ID, UUID) with no business meaning.

Polished Answer: The choice between natural and surrogate keys affects long-term maintainability:

- **Natural Keys:**
 - Human-readable and meaningful.
-

- May change (email changes, name changes).
- Can be composite (multiple columns).
- Examples: Email address, ISBN, passport number.
- **Risk:** Business rules change, "unique" attributes may not stay unique (email reused after deletion).

Surrogate Keys:

- System-generated, artificial identifiers.
- Never change (stable).
- Compact and efficient for indexing.
- No business meaning.
- Examples: Auto-increment ID, UUID, GUID.
- **Risk:** Data duplication possible if not combined with a natural unique constraint.

Best Practice: Use surrogate keys as primary keys (stability, performance) but add UNIQUE constraints on natural keys (business integrity).

TL;DR: Natural = business data, meaningful but changeable. Surrogate = system-generated, stable but no meaning. Use surrogate + unique on natural.

Keywords: Key types, Primary keys, Data stability, Database design, Business identifiers, Index performance

Q70: What is an N+1 query problem?

Answer: The N+1 query problem occurs when an application retrieves a list of entities and then makes an additional query for each entity to fetch related data, resulting in N+1 queries (1 initial + N per entity) instead of 1 optimized query.

Polished Answer: The N+1 problem is a common ORM performance issue:

- **What happens:**
- Loop: For each order (N = 100), run a separate query to fetch the customer.
- Total: 101 queries instead of 1 or 2.
- **Why it's problematic:**
- Scales badly--more records = more queries.
- Poor resource utilization.

Solutions:

- **Eager Loading:** Use JOINS to fetch all data in one query.
-

- **Batch Fetching:** Load related data for all parents in one query (WHERE parent_id IN (...)).
- **JOIN FETCH (in Hibernate):** Force a single query with JOIN.
- **Lazy loading with batch size:** Reduce number of queries by batching.

Prevention: Profile queries, understand ORM loading strategies, and use appropriate fetching strategies.

TL;DR: N+1 = fetching list + separate query per item. Solution: use JOINS or batch fetching to reduce queries.

Keywords: ORM performance, Eager loading, Lazy loading, Query optimization, Database round-trips, Application performance

Q71: What is a partial index?

Answer: A partial index is an index created on only a subset of rows in a table, based on a WHERE condition.

Polished Answer: Partial indexes optimize queries on specific value subsets:

- **How it works:**
- Reduces index size and maintenance overhead.
- Created with a predicate: `CREATE INDEX idx_active_users ON Users (Email) WHERE Status = 'Active';`

Benefits:

- Smaller index = less storage, faster writes.
- Query optimizer uses it only when query conditions match.
- Ideal for tables where only a small subset of data is frequently queried.

Use cases:

- Indexing only active records in a table with many inactive records.
- Indexing records from the last 30 days.
- Indexing rows where `deleted_at IS NULL`.

Limitation: Query conditions must match the index predicate for the optimizer to use it.

TL;DR: Partial index = index on subset of rows (WHERE clause). Smaller, faster, but only used for matching queries.

Keywords: Index optimization, Storage efficiency, Query predicates, Selective indexing, Database performance, Index size

Q72: What is the purpose of a transaction ID?

Answer: A transaction ID is a unique identifier assigned to each transaction in a DBMS. It is used to track transaction operations, order transactions, and manage concurrency and recovery.

Polished Answer: Transaction IDs are fundamental to transaction management:

- **Purpose:**
- Track which transaction modified which data.
- Order transactions in timestamp-based concurrency control.
- Coordinate rollback and recovery operations.

Uses:

- **Concurrency Control:** Determine transaction order (older vs newer).
- **Lock Management:** Track which transaction holds which locks.
- **Crash Recovery:** Identify committed vs uncommitted transactions.
- **MVCC:** Version data with the transaction ID that created it.
- **Deadlock Detection:** Identify the transactions involved in a deadlock.

Generation: The system assigns increasing IDs (often 64-bit integers) to transactions in their start order.

TL;DR: Transaction ID = unique identifier for each transaction. Used for tracking, ordering, concurrency control, and recovery.

Keywords: Transaction management, Concurrency control, Lock tracking, Crash recovery, MVCC, System-generated IDs

Q73: What is an upsert operation?

Answer: An upsert (UPDATE + INSERT) operation either inserts a new row if it doesn't exist or updates the existing row if it does. It's a combined operation that ensures a record with a given key exists with the specified values.

Polished Answer: Upsert is a convenient atomic operation:

- **Databases' syntax:**
- PostgreSQL: `INSERT INTO table (id, name) VALUES (1, 'John') ON`

```
CONFLICT (id) DO UPDATE SET name = 'John';
```

- SQL Server: MERGE INTO table USING ... WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...;

Benefits:

- Atomic--no separate check-then-insert/update steps.
- Eliminates race conditions (two processes both deciding to insert).
- Cleaner code--one statement instead of multiple.

Use cases:

- Synchronizing data from external sources.
- Maintaining counters (increment if exists, insert if not).
- Updating records that may or may not exist yet.

TL;DR: Upsert = INSERT if new, UPDATE if exists. Atomic, prevents race conditions. Syntax varies by database.

Keywords: INSERT OR UPDATE, ON CONFLICT, ON DUPLICATE KEY, MERGE, Atomic operations, Data synchronization

Q74: What is database sharding?

Answer: Sharding is a database partitioning technique that distributes data across multiple servers (shards), where each shard holds a portion of the data. It's a form of horizontal partitioning.

Polished Answer: Sharding is essential for massive-scale databases:

- **Sharding Strategies:**
- **Range-based Sharding:** Shards hold specific value ranges (e.g., users A-M, N-Z). Good for range queries but may create hotspots.
- **Directory-based Sharding:** Lookup service maps shard key to shard location. Flexible but adds lookup overhead.

Benefits:

- Horizontal scalability (add more shards as data grows).
- Improved query performance (each shard handles less data).
- Geographic distribution of data.

Challenges:

- Complexity: Cross-shard queries are difficult.
- Joins across shards are expensive or impossible.
- Data rebalancing when adding/removing shards.
- Distributed transaction management.
- Application must be shard-aware.

Shard Key Selection: Critical decision. Poor key choice leads to uneven distribution (hotspots).

TL;DR: Sharding = horizontal partitioning across servers. Benefits: scalability, performance. Challenges: cross-shard queries, complexity.

Keywords: Horizontal scaling, Data distribution, Shard key, Cross-shard queries, Distributed databases, Scalability

Q75: What is the difference between a hard delete and a soft delete?

Answer:

- **Hard Delete:** Permanently removes the record from the database.
- **Soft Delete:** Marks the record as deleted (using a flag or timestamp) but retains it in the database.

Polished Answer: Soft deletes are widely used in production systems:

- **Hard Delete:**
- Cannot be recovered without backup.
- Saves storage space.
- Cascading deletes remove related records.
- Performance degrades with large DELETE operations.

Soft Delete:

- Adds a column like `deleted_at` (TIMESTAMP) or `is_deleted` (BOOLEAN).
- Records remain in the table but are excluded from queries.
- Queries must filter: `WHERE deleted_at IS NULL`.
- Enables data recovery and audit history.
- Consumes more storage.
- Requires application or database-level filtering (views help).

When to use soft delete:

- Legal/compliance requirements (retain data for X years).
- User account deactivation (reactivation possible).
- Audit trail requirements.
- Referential integrity (references to deleted records remain valid).

TL;DR: Hard delete = permanent removal. Soft delete = mark as deleted but keep. Soft delete enables recovery and audit.

Keywords: Data deletion, Data recovery, Audit trails, Logical deletion, Data retention, Database design

Q76: What is a database trigger? What are its types?

Answer: A trigger is a special type of stored procedure that automatically executes in response to specific database events (INSERT, UPDATE, DELETE) on a table.

Types of triggers:

- **BEFORE trigger:** Executes before the operation.
- **AFTER trigger:** Executes after the operation.
- **INSTEAD OF trigger:** Replaces the operation entirely.

Polished Answer: Triggers automate database responses to events:

- **By Timing:**
- **AFTER Trigger:** Runs after the operation. Used for logging, auditing, cascading changes.
- **INSTEAD OF Trigger:** Runs in place of the operation. Used on views to handle non-updateable views.

By Event:

- **INSERT Trigger:** Fires on new row insertion.
- **UPDATE Trigger:** Fires when rows are updated.
- **DELETE Trigger:** Fires when rows are deleted.

Scope:

- **Row-level:** Fires once per affected row.
- **Statement-level:** Fires once per SQL statement (regardless of rows affected).

Use cases: Auditing, maintaining derived values, enforcing complex business rules, data validation.

TL;DR: Trigger = automatic response to table events. Types: BEFORE/AFTER/INSTEAD OF,

row/statement level.

Keywords: Event-driven, Database automation, Auditing, Data validation, Business rules, DML operations

Q77: What is a sequence in SQL?

Answer: A sequence is a database object that generates a series of unique numbers, often used for generating primary key values.

Polished Answer: Sequences are efficient ID generators:

- **Characteristics:**
- Used for surrogate keys.
- Fast--no table locking required.
- Supports increment, start value, min/max values, cycling.

Syntax (PostgreSQL):

sql

```
javascript.snippet                                JAVASCRIPT

CREATE SEQUENCE order_id_seq START WITH 1 INCREMENT BY 1 MAXVALUE 1000000; -- Usage: SELECT nextval('order_id_seq');
-- Or: CREATE TABLE Orders (    OrderID BIGINT DEFAULT nextval('order_id_seq'),
...);
```

- **vs Auto-increment:**
- Offer more control (increment by 2, cycling, caching).
- Auto-increment is a table column property; simpler but less flexible.

TL;DR: Sequence = number generator for unique IDs. More flexible than auto-increment. Used for surrogate keys.

Keywords: ID generation, Surrogate keys, Number series, Auto-increment, Database objects, Primary keys

Q78: What is the difference between an ORM and raw SQL?

Answer:

- **ORM (Object-Relational Mapping):** Programming technique that maps database tables

to objects in application code. Queries are written in the programming language.

- **Raw SQL:** Writing SQL queries directly in the application code.

Polished Answer: Each approach has strengths and weaknesses:

- **ORM:**
- Database-agnostic (mostly)--switch databases with minimal code changes.
- Automatic SQL generation for CRUD operations.
- Protection against SQL injection (parameterization built-in).
- Caching and connection pooling built-in.

Disadvantages:

- Performance overhead--generated SQL may be inefficient.
- N+1 query problem if not managed carefully.
- Complex queries are harder to write.
- Learning curve for ORM framework.

Examples: Hibernate, Sequelize, Django ORM, Entity Framework.

Raw SQL:

- **Advantages:**
- Better performance for complex queries.
- Leverages database-specific features.

Disadvantages:

- SQL injection risk if not parameterized.
- Database-dependent (portability issues).
- More code to maintain.
- Manual handling of object mapping.

TL;DR: ORM = abstraction over SQL (easy, portable, slower). Raw SQL = direct control (fast, flexible, risky). Use both where appropriate.

Keywords: Object mapping, Query generation, SQL injection, Portability, Performance, Development efficiency

Q79: What is connection pooling?

Answer: Connection pooling is a technique where a pool of reusable database connections is maintained, allowing applications to reuse existing connections instead of creating new ones for each request.

Polished Answer: Connection pooling dramatically improves application performance:

- **Why needed:** Creating a database connection is expensive (TCP handshake, authentication, session setup). Doing this per request adds significant overhead.
- **How it works:**
 - Application requests a connection from the pool.
 - After use, the connection is returned to the pool (not closed).
 - Pool handles connection lifecycle: creates new ones as needed, removes broken ones, enforces limits.

Benefits:

- Reduced connection setup overhead.
- Better resource management (max connection limits).
- Improved application response time.
- Connection reuse across requests.

Configuration: Pool settings include: min/max connections, idle timeout, connection validation query, max wait time.

TL;DR: Connection pool = reusable DB connections. Reduces setup overhead. Config: min/max connections, timeouts.

Keywords: Performance optimization, Resource management, Connection management, Application scaling, Database connectivity, Pool configuration

Q80: What is a write-ahead log (WAL)?

Answer: Write-ahead logging is a protocol where all database modifications are written to a log BEFORE they are written to the actual data files. This ensures data durability and supports crash recovery.

Polished Answer: WAL is fundamental for database durability:

- **How it works:**
 - The WAL entry is flushed to disk.
 - Only then is the actual data file modified.
 - On commit, a commit record is written to the WAL.
- **Benefits:**
 - **Durability:** Committed transactions survive failures.

- **Performance:** Sequential log writes are faster than random data file writes.

Recovery Process:

- On startup, the DBMS reads the WAL.
- Transactions with commit records are REDONE (applied to data files).
- Transactions without commit records are UNDONE (discarded).

Log management: Log files are periodically truncated or archived (checkpoints, log rotation).

TL;DR: WAL = log changes before applying to data. Ensures durability and crash recovery.
Sequential writes = performance benefit.

Keywords: Crash recovery, Data durability, Sequential logging, Transaction logging,
Database consistency, REDO/UNDO

Q81: What is a database lock escalation?

Answer: Lock escalation occurs when the DBMS automatically converts many fine-grained locks (row-level) into fewer coarse-grained locks (table-level) to reduce memory overhead and improve performance.

Polished Answer: Lock escalation balances resource usage and concurrency:

- **How it works:**
- The system converts all row-level locks to a single table-level lock.
- This reduces memory usage for lock management.

Trade-offs:

- **Benefit:** Less memory for lock tracking, fewer lock manager operations.
- **Cost:** Reduced concurrency--the entire table becomes locked, blocking other transactions from accessing any rows.

Triggers:

- Threshold of locks acquired (e.g., 5,000 row locks).
- Lock memory limits reached.
- Usually automatic and not directly controlled by application.

Management: DBAs can disable or tune escalation thresholds. Applications should avoid holding many locks.

TL;DR: Lock escalation = row locks → table lock. Saves memory but reduces concurrency.
Automatic threshold-based.

Keywords: Lock management, Row-level locks, Table-level locks, Concurrency trade-offs, Lock thresholds, Resource optimization

Q82: What is the purpose of a foreign key constraint with ON DELETE CASCADE?

Answer: ON DELETE CASCADE is a foreign key action that automatically deletes child records when the parent record is deleted, maintaining referential integrity.

Polished Answer: Cascade actions define what happens to child records when parent records are modified:

- **ON DELETE CASCADE:**
- Example: Delete a customer all their orders are also deleted.

Other actions:

- **ON DELETE SET NULL:** Sets child's foreign key to NULL.
- **ON DELETE SET DEFAULT:** Sets child's foreign key to its default value.
- **ON DELETE RESTRICT:** Prevents parent deletion if child rows exist.
- **ON DELETE NO ACTION:** Similar to RESTRICT (database-specific behavior).

Use cases:

- **CASCADE:** When child records have no meaning without parent.
- **SET NULL:** When child can exist without parent (nullable relationship).
- **RESTRICT:** When preventing deletion of referenced records is important.

Caution: CASCADE can unintentionally delete large amounts of data. Use deliberately.

TL;DR: ON DELETE CASCADE = auto-delete children when parent deleted. Other options: SET NULL, SET DEFAULT, RESTRICT.

Keywords: Referential integrity, Cascade actions, Foreign key constraints, Data relationships, Automatic deletion, Parent-child tables

Q83: What is the difference between a database and a data warehouse?

Answer:

- **Database (OLTP):** Stores current, operational data for daily transactions. Optimized for

fast writes and reads of individual records.

- **Data Warehouse (OLAP):** Stores historical, aggregated data from multiple sources for analysis and reporting. Optimized for complex queries and aggregations.

Polished Answer: Databases and data warehouses serve complementary roles:

- **Database (Operational):**
- **Data:** Current, detailed operational data.
- **Schema:** Normalized (reduce redundancy for fast writes).
- **Queries:** Simple, short, high-concurrency.
- **Users:** Application users (customers, employees).
- **Storage:** Current data (days to months).
- **Examples:** MySQL, PostgreSQL, SQL Server.

Data Warehouse:

- **Purpose:** Support business intelligence and analytics.
- **Data:** Historical, aggregated data from multiple sources.
- **Schema:** Denormalized (star schema, snowflake schema).
- **Queries:** Complex, long-running, analytical.
- **Users:** Analysts, managers, data scientists.
- **Storage:** Years of historical data.
- **Examples:** Amazon Redshift, Google BigQuery, Snowflake.

TL;DR: Database = current operational data (fast writes). Data Warehouse = historical analytical data (complex queries).

Keywords: OLTP vs OLAP, Data analysis, Schema design, Operational vs analytical, Reporting, Data storage

Q84: What is a full-text search index?

Answer: A full-text index is a special type of index that enables efficient searching of text data, supporting keyword searches, phrase searches, and ranking of results by relevance.

Polished Answer: Full-text indexes enable sophisticated text search:

- **What it provides:**
 - Phrase matching ("exact phrase").
 - Prefix matching (word starts with).
 - Relevance ranking (most relevant documents first).
-

- Stop word filtering ("the", "is", "and").
- Stemming (search "running" also finds "run").

Database implementations:

- MySQL: FULLTEXT index with MATCH...AGAINST.
- PostgreSQL: Full-text search with tsvector and tsquery.
- SQL Server: Full-Text Catalog.

vs LIKE search:

- LIKE '%keyword%' does full table scan--slow for large tables.
- Full-text index pre-computes word lists--fast even on millions of rows.

Use cases: Search bars, document management, forum/comment search, product search.

TL;DR: Full-text index = specialized index for text search. Supports keywords, phrases, relevance ranking. Much faster than LIKE.

Keywords: Text search, Keyword matching, Relevance ranking, Stemming, Full-text search, Search optimization

Q85: What is a database migration?

Answer: Database migration is the process of modifying the database schema--adding, modifying, or removing tables, columns, indexes, and constraints--usually in a versioned, repeatable manner.

Polished Answer: Database migrations manage schema evolution:

- **What migrations include:**
- Adding/removing/modifying columns.
- Creating/modifying indexes.
- Adding/modifying constraints.
- Data transformations (migrate existing data).

Migration tools:

- Flyway, Liquibase (Java).
- Django migrations, Alembic (Python).
- Knex.js, Sequelize migrations (Node.js).

Best practices:

- **Versioned migrations:** Each migration has a version number.

- **Forward and backward migrations:** Can apply (up) or rollback (down).
- **Idempotent where possible:** Safe to run multiple times.
- **Test migrations:** Test against production-like data.
- **Backup before migration:** Critical for large changes.
- **Zero-downtime migrations:** Use techniques like expand-contract (add new, then remove old).

TL;DR: Migration = versioned schema changes. Tools: Flyway, Liquibase, Alembic. Best practices: versioned, testable, reversible.

Keywords: Schema evolution, Version control, Flyway, Liquibase, Database deployment, Schema changes

Q86: What is the difference between a unique index and a unique constraint?

Answer:

- **Unique Constraint:** A rule that ensures column values are unique. It's a logical constraint.
- **Unique Index:** A physical index that enforces uniqueness. It's a data structure.

Polished Answer: While related, they serve different purposes:

- **Unique Constraint:**
- Part of the schema definition.
- Can be referenced by foreign keys.
- In most databases, creating a unique constraint automatically creates a unique index.

Unique Index:

- Physical structure that enforces uniqueness.
- Speeds up lookups based on the indexed column(s).
- Cannot be referenced by foreign keys directly.
- Can be created independently of constraints.

Practical difference: In databases where both exist separately (like SQL Server), you can have a unique index without a unique constraint. Foreign keys require a unique constraint (not just an index). When both exist for the same column, they serve dual purposes: logical integrity (constraint) and physical access optimization (index).

TL;DR: Unique constraint = logical rule (can be FK target). Unique index = physical structure (speeds lookups). Most databases create both automatically.

Keywords: Uniqueness enforcement, Foreign key references, Index structure, Data integrity, Physical vs logical, Constraint implementation

Q87: What is a database snapshot?

Answer: A database snapshot is a read-only, point-in-time copy of a database that captures the database state at the moment of snapshot creation.

Polished Answer: Database snapshots provide instant point-in-time views:

- **How it works (Copy-on-Write):**
- When the source database modifies a page, the original page is copied to the snapshot before modification.
- Snapshot reads unchanged pages from the source and changed pages from the snapshot.

Benefits:

- Instant creation (no full data copy).
- Minimal storage overhead initially.
- Useful for reporting against a point-in-time state.
- Can revert source database to snapshot state.

Limitations:

- Not a full backup--doesn't protect against source database loss.
- Storage grows as source database changes.
- Snapshot becomes stale over time.

Use cases: Before schema changes, before major data modifications, reporting consistency.

TL;DR: Snapshot = instant point-in-time view using copy-on-write. Not a backup. Useful for reporting and revert points.

Keywords: Point-in-time, Copy-on-write, Read-only view, Instant creation, Database states, Reporting consistency

Q88: What is the difference between relational and non-relational databases?

Answer:

- **Relational databases:** Store data in structured tables with rows and columns. Support

relationships using foreign keys. Enforce integrity using constraints.

- **Non-relational databases (NoSQL):** Store data in flexible formats (key-value, document, column, graph). No fixed schema.

Polished Answer: The choice between relational and non-relational affects architecture:

- **Relational (SQL):**
- **Data model:** Tables with rows and columns.
- **Relationships:** Foreign keys, JOINS.
- **Scaling:** Vertically (bigger server) primarily.
- **Transactions:** ACID compliant.
- **Examples:** MySQL, PostgreSQL, Oracle, SQL Server.
- **Best for:** Structured data, complex queries, data integrity, financial systems.

Non-relational (NoSQL):

- **Schema:** Flexible, dynamic.
- **Data model:** Documents (MongoDB), key-value (Redis), column (Cassandra), graph (Neo4j).
- **Relationships:** Denormalized data, application-level joins.
- **Scaling:** Horizontally (more servers).
- **Transactions:** BASE (Basically Available, Soft state, Eventually consistent).
- **Examples:** MongoDB, Redis, Cassandra, DynamoDB, Neo4j.
- **Best for:** Unstructured data, rapid development, massive scale, high throughput.

TL;DR: Relational = structured tables, ACID, vertical scaling. NoSQL = flexible schema, horizontal scaling, eventual consistency.

Keywords: Data models, Schema design, Scalability, ACID vs BASE, Structured vs unstructured, Database types

Q89: What is a graph database?

Answer: A graph database is a NoSQL database that stores data as nodes (entities) and edges (relationships), optimized for traversing and analyzing complex relationships.

Polished Answer: Graph databases excel at relationship-centric queries:

- **Data model:**
 - **Edges:** Relationships between nodes (directed or undirected).
 - **Properties:** Attributes on nodes and edges.
-

Optimized for:

- Traversal of relationships (friend-of-friend queries).
- Path finding (shortest path).
- Pattern matching in connected data.

vs Relational DB: Relational databases require expensive recursive joins for deep relationships. Graph databases traverse relationships in constant time (follow a pointer).

Use cases:

- Social networks (friend recommendations).
- Fraud detection (transaction patterns).
- Recommendation engines.
- Supply chain networks.
- Knowledge graphs.

Examples: Neo4j, Amazon Neptune, OrientDB.

TL;DR: Graph DB = nodes + edges optimized for relationship traversal. Fast path finding. Use for social networks, fraud detection.

Keywords: Nodes, Edges, Relationship traversal, Path finding, Neo4j, Connected data

Q90: What is a document database?

Answer: A document database is a NoSQL database that stores data as self-contained documents (usually JSON or BSON). Each document can have a different structure.

Polished Answer: Document databases offer flexibility for hierarchical data:

- **Data model:**
- Each document can have different fields.
- Nested objects and arrays supported.
- Documents stored in collections (loosely equivalent to tables).

Benefits:

- Flexible schema--no migrations needed.
- Documents map naturally to objects in programming.
- Good for hierarchical/nested data.
- Horizontal scaling.

vs Relational:

- No fixed columns.
- No JOINS--related data embedded in the same document.
- Denormalization encouraged.

Use cases: Content management, user profiles, catalogs, IoT data.

Examples: MongoDB, CouchDB, Amazon DocumentDB.

TL;DR: Document DB = JSON-like documents, flexible schema, no JOINS. Good for hierarchical data and rapid development.

Keywords: JSON documents, Schema-less, NoSQL, Nested data, MongoDB, Flexible structure

Q91: What is the purpose of the EXPLAIN command?

Answer: The EXPLAIN command shows the execution plan for a SQL query, detailing how the database will execute the query--which indexes will be used, join types, and access methods.

Polished Answer: EXPLAIN is essential for query optimization:

- **What it shows:**
- Indexes used or considered.
- Join types (nested loop, hash join, merge join).
- Join order.
- Estimated row counts and costs.
- Sorting and grouping operations.

Usage:

sql

```

javascript.snippet
JAVASCRIPT

EXPLAIN SELECT FROM Users WHERE Email = 'test@example.com';-- Or analyze for actual execution
:EXPLAIN ANALYZE SELECT FROM Users WHERE Email = 'test@example.com';

```

- **What to look for:**
- **High estimated rows** (inefficient filtering).
- **Inefficient join types** (cartesian joins, etc.).

- **Sorting operations** (temporary table creation).

Benefits: Identify missing indexes, optimize query structure, understand performance bottlenecks.

TL;DR: EXPLAIN = shows query execution plan. Reveals index usage, join types, scan methods. Essential for query optimization.

Keywords: Query optimization, Execution plans, Index analysis, Performance tuning, Access methods, Query analysis

Q92: What is the difference between a unique constraint and a primary key constraint?

Answer:

- **Primary Key:** Uniquely identifies each row. Cannot contain NULL. Only one per table.
- **Unique Constraint:** Ensures unique values. Can contain NULL. Multiple per table.

Polished Answer: Both enforce uniqueness, but primary keys have additional properties:

- **Primary Key:**
- Cannot have NULL values (both NOT NULL and UNIQUE).
- Only one per table.
- Often used as foreign key reference in other tables.
- Typically creates a clustered index (in many databases).

Unique Constraint:

- Ensures all values are distinct.
- Can have NULL values (usually one NULL per column, MySQL allows multiple).
- Multiple unique constraints per table.
- Can be used as foreign key target (in some databases, like PostgreSQL).
- Creates a unique index.

Key insight: The primary key is ONE special unique constraint designated as the main identifier. All other uniqueness rules use unique constraints.

TL;DR: Primary key = unique + NOT NULL + one per table (main identifier). Unique = unique + allows NULL + multiple per table.

Keywords: Uniqueness, NULL handling, Row identification, Foreign keys, Index creation, Constraint types

Q93: What is data integrity? What are its types?

Answer: Data integrity refers to the accuracy, consistency, and reliability of data in a database.

Types:

- **Entity Integrity:** Ensures each row is uniquely identified (primary keys).
- **Referential Integrity:** Ensures relationships between tables are maintained (foreign keys).
- **Domain Integrity:** Ensures values in a column are valid (data types, constraints).

Polished Answer: Data integrity is a fundamental database goal:

- **Entity Integrity:**
 - Primary key cannot be NULL.
 - Ensures each row is uniquely identifiable.

Referential Integrity:

- Foreign keys must match primary keys or be NULL.
- Prevents orphaned records.
- Maintains relationships between tables.

Domain Integrity:

- Values conform to defined domain (data type, length, format).
- Enforced by data types, CHECK constraints, NOT NULL.
- Example: Age must be integer and > 0.

User-Defined Integrity:

- Business-specific rules.
- Enforced by triggers, stored procedures.
- Example: Salary cannot decrease by more than 50%.

TL;DR: Data integrity = data accuracy and consistency. Types: Entity (unique rows), Referential (valid relationships), Domain (valid values), User-defined (business rules).

Keywords: Data accuracy, Data consistency, Primary keys, Foreign keys, Domain constraints, Business rules

Q94: What is database normalization vs denormalization?

Answer:

- **Normalization:** Process of organizing data to reduce redundancy and improve integrity.
- **Denormalization:** Process of combining tables to improve query performance by introducing redundancy.

Polished Answer: These are opposing strategies with different goals:

- **Normalization:**
- **Process:** Decompose tables into smaller, related tables.
- **Benefits:** Data consistency, less storage, easier updates.
- **Drawbacks:** More JOINS needed, potentially slower reads.
- **Use case:** OLTP systems with frequent writes.

Denormalization:

- **Goal:** Improve read performance.
- **Process:** Combine tables or add redundant data.
- **Benefits:** Faster queries, fewer JOINS.
- **Drawbacks:** Higher storage, risk of inconsistency, complex updates.
- **Use case:** OLAP systems, reporting, read-heavy applications.

Real-world: Most systems use a mix--normalized for transactional integrity, denormalized for reporting.

TL;DR: Normalization = reduce redundancy (write-optimized). Denormalization = add redundancy (read-optimized). Use based on workload.

Keywords: Data redundancy, Query performance, Write optimization, Read optimization, Schema design, Data integrity

Q95: What is the purpose of the VACUUM command in PostgreSQL?

Answer: VACUUM in PostgreSQL reclaims storage occupied by dead rows (rows that have been updated or deleted) and updates statistics used by the query planner.

Polished Answer: VACUUM is essential for PostgreSQL maintenance:

- **What it does:**
- Updates table statistics for the query planner.
- Prevents transaction ID wraparound.

- Can analyze tables for better query plans.

Why needed: PostgreSQL uses MVCC--UPDATE creates a new row version, old version is marked dead. Without VACUUM, dead rows accumulate, causing bloat.

Types:

- **VACUUM:** Reclaims space but doesn't return it to OS (keeps for future use).
- **VACUUM FULL:** Fully rewrites the table, returns space to OS (locks table).
- **VACUUM ANALYZE:** Also updates statistics.

AUTOVACUUM: PostgreSQL runs VACUUM automatically when enough rows become dead.

TL;DR: VACUUM = PostgreSQL cleanup. Reclaims dead row space, updates statistics.

AUTOVACUUM runs automatically.

Keywords: PostgreSQL maintenance, Dead rows, Storage reclamation, Query statistics, MVCC cleanup, Database bloat

Q96: What is a database deadlock vs livelock?

Answer:

- **Deadlock:** Two or more transactions are permanently blocked, each waiting for resources the other holds.
- **Livelock:** Transactions are not blocked but keep retrying and failing, never making progress.

Polished Answer: Both are concurrency problems but with different characteristics:

- **Deadlock:**
- No transaction can proceed.
- Requires detection and resolution (victim rollback).
- Example: A locks Row1, wants Row2; B locks Row2, wants Row1.

Livelock:

- Transactions repeatedly interact without making progress.
- Usually occurs with retry mechanisms--two transactions repeatedly collide and retry.
- No resource held indefinitely, but no work completed.
- Resolution: Random backoff delays.
- Example: Two transactions both try to acquire the same lock, both back off and retry simultaneously, colliding again.

Prevention:

- Deadlock: Lock ordering, timeouts, detection.
- Livelock: Random backoff, priority-based allocation.

TL;DR: Deadlock = circular waiting (permanent block). Livelock = repeated retry collisions (no progress, not blocked). Both require resolution.

Keywords: Concurrency problems, Lock contention, Retry mechanisms, Transaction management, Resource allocation, Lock ordering

Q97: What is the difference between ALTER TABLE and UPDATE?

Answer:

- **ALTER TABLE:** DDL command that modifies the table structure (add/remove columns, modify data types, add constraints).
- **UPDATE:** DML command that modifies data within existing rows.

Polished Answer: ALTER TABLE and UPDATE operate at different levels:

- **ALTER TABLE (DDL):**
- Operations: ADD COLUMN, DROP COLUMN, MODIFY COLUMN, ADD CONSTRAINT.
- Not transactional in all databases (MySQL commits implicitly).
- Affects all rows in the table.
- Example: ALTER TABLE Users ADD COLUMN Age INT;

UPDATE (DML):

- Changes DATA within existing structure.
- Operations: Modify values in existing columns.
- Transactional (can be rolled back).
- Affects only rows matching the WHERE condition.
- Example: UPDATE Users SET Age = 25 WHERE ID = 1;

Key insight: ALTER TABLE changes WHAT data can be stored (schema), UPDATE changes WHAT data IS stored (values).

TL;DR: ALTER = changes structure (schema, all rows). UPDATE = changes data values (specific rows). DDL vs DML.

Keywords: DDL vs DML, Schema changes, Data modifications, Table structure, Transaction

Q98: What is a covering index?

Answer: A covering index is an index that includes all columns needed to satisfy a query, eliminating the need to access the table for additional data.

Polished Answer: Covering indexes optimize query performance significantly:

- **How it works:**
- If all these columns exist in the index, the DBMS can answer the query using only the index.
- No bookmark lookup needed to access the table.

Example:

- Query: `SELECT Email FROM Users WHERE City = 'New York' AND Status = 'Active';`
- Index on (City, Status, Email) is a covering index because it contains all referenced columns.

Benefits:

- Eliminates table access (reduces I/O).
- Significantly faster queries.
- Index-only scan (shown in query plans as "Index Only Scan" or "Using index").

Trade-offs:

- Larger index size (more columns).
- More write overhead (must update more index entries).

Use when: Queries frequently access a specific set of columns, and write performance isn't critical.

TL;DR: Covering index = contains all query columns. No table lookup needed. Faster queries, larger index size.

Keywords: Index optimization, Query performance, Index-only scan, Storage trade-offs, Index design, Lookup elimination

Q99: What is a database trigger's INSTEAD OF type?

Answer: An INSTEAD OF trigger replaces the triggering operation entirely. Instead of performing the INSERT, UPDATE, or DELETE, the trigger's logic executes.

Polished Answer: INSTEAD OF triggers are particularly useful for views:

- **How it works:**

- The operation is NOT performed.
- Instead, the trigger's logic executes.
- The trigger must explicitly perform any required operations.

Primary use case:

- Making non-updateable views updateable.
- Complex views (with JOINS) cannot be directly updated.
- An INSTEAD OF trigger defines how the update should be applied to underlying tables.

Example:

sql

```
javascript.snippet JAVASCRIPT

CREATE VIEW CustomerOrders AS SELECT C.ID, C.Name, O.OrderID, O.Amount FROM Customers
CJOIN
  Orders O ON C.ID = O.CustomerID; CREATE TRIGGER UpdateCustomerOrders INSTEAD OF UPDATE
ON
  CustomerOrders FOR EACH ROW BEGIN   UPDATE Customers SET Name = NEW.Name WHERE ID = OLD
  ID;   UPDATE Orders SET Amount = NEW.Amount WHERE OrderID = OLD.OrderID; END;
```

- **Database support:** SQL Server, PostgreSQL, Oracle support INSTEAD OF triggers. MySQL doesn't support this type directly.

TL;DR: INSTEAD OF trigger = replaces operation entirely. Main use: making complex views updateable. Not supported in all databases.

Keywords: Trigger types, View updating, Operation replacement, Database automation, Complex views, DML operations

Q100: What is the purpose of the COMMIT and ROLLBACK commands?

Answer:

- **COMMIT:** Saves all changes made during the current transaction to the database

permanently.

- **ROLLBACK:** Reverses all changes made during the current transaction, restoring the database to its previous state.

Polished Answer: COMMIT and ROLLBACK are fundamental transaction controls:

- **COMMIT:**
- Once committed, changes cannot be undone.
- Releases locks held by the transaction.
- Fires any AFTER COMMIT triggers or deferred constraints.
- Example: `BEGIN TRANSACTION; UPDATE Accounts SET Balance = Balance - 100 WHERE ID = 1; UPDATE Accounts SET Balance = Balance + 100 WHERE ID = 2; COMMIT;`

ROLLBACK:

- Undoes all changes made in the transaction.
- Restores database to state before transaction began.
- Releases locks held by the transaction.
- Used on error or explicit cancellation.
- Example: `BEGIN TRANSACTION; DELETE FROM Orders WHERE CustomerID = 999; -- Oops, wrong customer! ROLLBACK;`

Impact on ACID:

- Atomicity: Both commands ensure all-or-nothing execution.
- Durability: COMMIT makes changes permanent, even through crashes.
- Consistency: ROLLBACK ensures the database returns to a consistent state.

Auto-commit behavior: Many databases have auto-commit enabled by default (each statement is its own transaction).

TL;DR: COMMIT = make changes permanent (durable). ROLLBACK = undo changes (atomic). Essential for ACID compliance.

Keywords: Transaction control, Data persistence, Atomicity, Durability, Data consistency, Transaction management

QUICK REFERENCE: KEY TERMS MAPPING

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability
DDL	Data Definition Language (CREATE, ALTER, DROP)
DML	Data Manipulation Language (SELECT, INSERT, UPDATE, DELETE)
DCL	Data Control Language (GRANT, REVOKE)
TCL	Transaction Control Language (COMMIT, ROLLBACK, SAVEPOINT)
1NF	First Normal Form - atomic values
2NF	Second Normal Form - no partial dependencies
3NF	Third Normal Form - no transitive dependencies
BCNF	Boyce-Codd Normal Form - every determinant is candidate key
OLTP	Online Transaction Processing - transactional systems
OLAP	Online Analytical Processing - analytical systems
MVCC	Multi-Version Concurrency Control
WAL	Write-Ahead Logging
2PC	Two-Phase Commit
CAP	Consistency, Availability, Partition Tolerance
ERD	Entity-Relationship Diagram
CTE	Common Table Expression
ORM	Object-Relational Mapping
DBA	Database Administrator
S Lock	Shared Lock (read lock)
X Lock	Exclusive Lock (write lock)
N+1	N+1 Query Problem (ORM inefficiency)